

UNIVERSITATEA TEHNICĂ „Gheorghe Asachi” din IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DOMENIUL: CALCULATOARE ȘI TEHNOLOGIA
INFORMAȚIEI
SPECIALIZAREA: TEHNOLOGIA INFORMAȚIEI

Platformă de gestiune a informațiilor despre vehicule, folosind blockchain

LUCRARE DE DIPLOMĂ

Coordonator științific
Ș.l.dr.inf. Tiberius Dumitriu

Absolvent
Matei Rareș

Iași, 2024

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII
LUCRĂRII DE DIPLOMĂ

Subsemnatul(a) MATEI RAREȘ,

legitimat(ă) cu CI , seria NZ , nr. 037153 , CNP 5010223271540

autorul lucrării

PLATFORMĂ DE GESTIUNE A INFORMAȚIILOR DESPRE VEHICULE,
FOLOSIND BLOCKCHAIN

elaborată în vederea susținerii examenului de finalizare a studiilor de licență, programul de studii CALCULATOARE ȘI TEHNOLOGIA INFORMATIEI organizat de către Facultatea de Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi” din Iași, sesiunea IULIE 2024 a anului universitar 2023-2024 , luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 – Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a convențiilor internaționale privind drepturile de autor.

Data

27.06.2024

Semnătura



Iași, 2024

Cuprins

Introducere	1
Capitolul I. Concepte, tehnologii, arhitectură	3
I.1. Noțiuni teoretice	3
I.1.1. <i>Blockchain</i>	3
I.1.2. <i>Smart contracts</i>	3
I.1.3. <i>Rețele neuronale</i>	5
I.2. Tehnologii utilizate	5
I.2.1. <i>Angular</i>	5
I.2.2. <i>Python</i>	6
I.2.3. <i>Solidity</i>	7
I.2.4. <i>Ganache</i>	7
I.3. Arhitectura aplicației	8
I.3.1. <i>Nivelul de prezentare</i>	8
I.3.2. <i>Nivelul de aplicație</i>	10
I.3.3. <i>Nivelul de date</i>	12
Capitolul II. Funcționalitate și implementare	13
II.1. Funcționalitate	13
II.1.1. <i>Utilizator neautentificat</i>	13
II.1.2. <i>Utilizator autentificat</i>	15
II.1.3. <i>Manager de utilizatori</i>	16
II.2. Implementare	17
II.2.1. <i>Nivelul de prezentare</i>	17
II.2.2. <i>Nivelul de aplicație</i>	18
II.2.3. <i>Smart contract</i>	26
II.2.4. <i>Rețele neuronale</i>	29
Concluzii	33
Bibliografie	35
Anexe	38

Platformă de gestiune a informațiilor despre vehicule, folosind blockchain

Matei Rareș

Rezumat

În prezent, oamenii se pot întâlni cu necesitatea de a se deplasa rapid dintr-un punct în altul, fie în interes personal, fie profesional. Printre variantele disponibile este deținerea unui

autovehicul, care poate fi o alegere practică în economisirea timpului și a banilor pe termen lung, indiferent dacă este vorba despre un vehicul second-hand cu motorizare clasică sau electrică. O problemă majoră în alegerea unui model poate fi lipsa informațiilor de încredere despre autoturism, deoarece modificările neautorizate sau chiar ilegale pot afecta atât siguranța cât și cheltuielile clientului. De multe ori, sursa principală de informare este chiar deținătorul sau dealer-ul, care, cu toate că este conștient de posibilele probleme tehnice, ar putea să minimalizeze sau să evite comunicarea acestora pentru profit personal.

Dificultatea întâlnită de oameni pentru a obține informații de încredere în procesul de achiziționare a autovehiculelor, fapt ce poate duce la decizii greșite, riscuri de siguranță sau costuri adiționale, mai ales în cazul vehiculelor second-hand, au dus la crearea unei aplicații care să înlăture o parte din aceste neajunsuri, fiind propuse și câteva metode posibile pentru creșterea încrederii clientului în alegerea făcută.

Alegerea acestei teme este motivată și de necesitatea de a reduce timpul și costurile asociate eventualelor probleme viitoare ale vehiculului, prin utilizarea unor tehnici de creștere a securității prelevării și păstrării informației. În plus, s-a dorit să se exploreze metodele posibile de utilizare a tehnologiilor de tip blockchain într-o soluție practică care să ușureze modul în care utilizatorul își poate spori încrederea în informațiile primite.

Acest lucru s-a realizat prin implementarea unei platforme care să asigure o evidență corectă a istoricului, evenimentelor și a datelor unui automobil pentru a le furniza într-o manieră transparentă și de încredere potențialilor clienți.

În introducere, este descrisă în detaliu tematica, abordând contextul și importanța acesteia, după care sunt analizate critic alte soluții existente care încearcă să rezolve problemele enumerate.

Capitolul 1 oferă o prezentare a motivelor din spatele alegerii tehnologiilor utilizate, explicând cum aceste decizii au contribuit la dezvoltarea arhitecturii aplicației.

Capitolul 2 se concentrează pe implementarea aplicației, oferind o descriere amănunțită a fiecărui pas din procesul de dezvoltare.

La final, în secțiunea dedicată concluziilor sunt evidențiate rezultatele obținute și diverse modalități de îmbunătățire a soluției actuale urmată mai apoi de bibliografia care include referințele utilizate.

Tehnologii principale folosite, alese pentru o implementare fluidă și predispusă dezvoltării sunt următoarele: Angular [1], Python [2], Solidity [3] cu Ganache [4].

Din punct de vedere software, pentru a executa programul este nevoie de versiunea 3.10 de Python împreună cu diverse biblioteci, Npm [5], Node.js [6] și Angular cli pentru Angular, Solc (Solidity Compiler) versiunea 0.8.0 [7] sau una superioară pentru Solidity și aplicația Ganache.

Din punct de vedere hardware, aplicația a fost testată pe un dispozitiv cu sistem de operare Windows 10, 16 GB RAM și un procesor AMD Ryzen 5 4600H.

Introducere

Ritmul alert al vieții moderne ne impune adesea necesitatea de a ne deplasa rapid, fie din motive personale, fie profesionale, către destinații îndepărtate, unde timpul devine esențial. Pierderea de timp și efort asociată cu metodele de transport mai lente poate afecta negativ productivitatea și calitatea vieții. Din acest motiv, utilizarea unui autovehicul personal devine o soluție eficientă, oferind deplasări rapide, flexibile și confortabile, esențiale pentru cei care călătoresc frecvent, fie că este vorba de drumuri zilnice la locul de muncă sau de călătorii ocazionale în afara orașului.

În acest context, achiziționarea unui vehicul devine o opțiune avantajoasă, iar o alternativă profitabilă este cea a mașinilor second-hand deoarece oferă un raport bun între preț și utilitate, făcându-le accesibile unui public mai larg. Acestea pot fi găsite într-o stare bună, la prețuri mult mai accesibile comparativ cu cele noi, și pot oferi același nivel de confort și eficiență.

În momentul achiziționării unui autovehicul, fie că este nou sau second-hand, informațiile esențiale despre starea și istoricul obiectului în cauză sunt deținute de dealer sau de proprietar. Problema principală este că, din motive precum obținerea unui profit personal, proprietarul mașinii poate alege să nu împărtășească toate informațiile, mai ales care ar pune produsul într-o lumină nefavorabilă. De exemplu, o piesă care a fost folosită ca o soluție temporară, în viitor, poate cauza probleme serioase. Astfel, lipsa detaliilor expune clienții la riscul de a achiziționa un produs cu defecte, care necesită reparații costisitoare sau care nu sunt sigure pentru utilizare. Aceste tipuri de situații generează nu doar sentimente negative ci și pierderi financiare, de timp sau în cazuri extreme, de viați umane.

Tema are ca scop oferirea unei platforme ce poate ajuta la obținerea unor informații veridice și complete despre o mașină înainte de a o achiziționa sau de a o pune spre vânzare, fără a depinde de o persoană fizică sau de un dealer, pentru a crește gradul de încredere în acea tranzacție. În cazul achiziționării, acest motiv este susținut de reducerea timpului și a costurilor asociate posibilelor probleme viitoare cauzate de absența datelor complete asupra mașinii. În cazul vânzătorului, acesta poate beneficia de o vedere de ansamblu corectă a vehiculului fapt care poate duce la o tranzacție ușoară și plăcută, făcând să crească indirect și gradul de încredere al clientului.

Scopul lucrării, care abordează această temă, este de a implementa o platformă care să ofere acces la informații într-o manieră transparentă, eliminând necesitatea unui intermediar care deține detaliile vehiculului. Această platformă trebuie să permită utilizatorilor să acceseze istoricul complet al mașinii, inclusiv reparațiile efectuate, accidentele anterioare și orice alte informații relevante care pot influența decizia de cumpărare. Astfel, cumpărătorii ar putea face alegeri informate, bazate pe date corecte și complete, reducând riscurile asociate achiziției de vehicule second-hand.

O astfel de platformă ar reprezenta un instrument valoros nu doar pentru cumpărători, ci și pentru persoanele de bună-credință, care doresc să își vândă vehiculele în mod transparent și

corect. Prin promovarea transparenței și a accesului liber la informații, platforma ar putea contribui la crearea unui mediu de piață mai echitabil și mai sigur pentru toți participanții.

Deoarece lipsa de informații legate de istoricul mașinilor vândute, noi sau second-hand, este o problemă prezentă de mulți ani, s-au dezvoltat numeroase website-uri care oferă servicii de informare a utilizatorului în legătură cu un vehicul identificat prin seria de șasiu, metodă adoptată și de aplicația în discuție. Câteva din aceste site-uri găsite pe internet sunt Autodna [8], Rarom [9] sau Vincheck [10], care reprezintă opțiuni gratuite pentru verificarea informațiilor despre un autovehicul. În ciuda acestui fapt, aceste resurse oferă o gamă mai limitată de date, care nu pot crea o imagine completă de ansamblu asupra vehiculului.

O alternativă viabilă este CarVertical [11], care furnizează un set impresionant de informații legate de starea și istoricul autovehiculului, inclusiv detalii despre eventualele accidente sau furturi prin care a trecut mașina. Pe de altă parte, accesul la aceste detalii este disponibil doar contra cost, ceea ce poate determina unele persoane să fie mai puțin dispuse să investească în verificarea completă a unui autoturism înainte de achiziție sau vânzare. Acest aspect poate conduce utilizatorul să folosească mai rar aceste servicii sau să analizeze atent necesitatea cheltuielilor pentru o eventuală viitoare achiziție. Proiectul își propune să diminueze aceste limitări prin oferirea unui mediu transparent, ușor de accesat și de interacționat, pentru o documentare cât mai eficientă în perspectiva unei posibile tranzacții.

Capitolul I. Concepte, tehnologii, arhitectură

În acest capitol, sunt prezentate în detaliu conceptele fundamentale în înțelegerea mecanismului prin care lucrarea a fost creată. Tot aici sunt aduse în atenție tehnologiile utilizate și modul în care acestea sunt integrate și aplicate în cadrul arhitecturii, oferind o perspectivă amănunțită asupra modului în care contribuie fiecare componentă la desfășurarea a sistemului.

I.1. Noțiuni teoretice

I.1.1. Blockchain

În acest studiu se propune utilizarea tehnologiei blockchain, în care datele *tranzacțiilor* (acțiunile petrecute pe blockchain) sunt transparente, dar tehnicile de *hashing* (operație matematică unidirecțională care transformă datele de intrare într-un șir fix de caractere unice) împiedică utilizatorul să vadă în clar informația. Datorită acestor algoritmi, se folosesc *contracte inteligente* (*smart contracts* – engl.) care reprezintă părți de cod executate pe blockchain și au acces la starea acestuia [12].

Blockchain-ul [13] este cunoscut ca o rețea descentralizată de noduri ce mențin un registru distribuit care înregistrează toate tranzacțiile și schimbările de stare [14]. Fiecare nod al rețelei are o copie sau o parte a acestui registru iar starea blockchain-ului este reprezentată de valorile curente ale tuturor conturilor și variabilelor stocate în rețea.

În ciuda faptului că tehnologia blockchain este adesea asociată cu criptomonede, aceasta poate fi utilizată într-o gamă diversă de domenii, cum ar fi rețelele de socializare, sănătate, finanțe sau educație. Una dintre cele mai des utilizate aplicații ale acestei tehnologii este în cadrul lanțurilor de aprovizionare.

Un blockchain poate fi folosit și ca o bază de date securizată și transparentă datorită naturii sale descentralizate și imutabile, caracteristici care sunt discutate în diverse lucrări [15], [16], [17]. Unul dintre cele mai populare utilizări ale blockchain-ului în acest sens este pe platforma Ethereum [18], care este cunoscută pentru abilitatea sa de a menține contracte inteligente.

I.1.2. Smart contracts

Smart contracts sunt programe stocate pe blockchain, care se execută automat odată ce anumite condiții sunt îndeplinite. Ele pot fi scrise folosind diferite limbaje de programare orientate pe contract, unul dintre acestea fiind Solidity. O caracteristică principală a acestor contracte este imutabilitatea, deci odată stocate pe rețea, nu mai pot fi modificate [19], decât dacă s-a intenționat inițial un mecanism de actualizare [20].

În lucrarea aceasta nu sunt implementate astfel de metode, fapt ce asigură consecvența și fiabilitatea contractului. Contractele pot include funcții de citire sau scriere a datelor pe blockchain. Aceste funcții sunt apelate de tranzacții inițiate de entități externe sau alte contracte. Atunci când un contract inteligent trebuie să preia date de pe blockchain, poate să facă o referire simplă la variabilele relevante sau să facă apeluri către alte contracte care dețin informațiile dorite.

Prin intermediul acestor *smart contracts*, este posibilă crearea de *aplicații descentralizate* (*dApps* - engl.) [21] care interacționează cu baze de date distribuite. Un exemplu de utilizare a contractelor inteligente este prezentat de către Fabian Dietrich, Daniel Palm și Luis Louw [22], care observă o posibilă îmbunătățire a securității în domeniul lanțurilor de aprovizionare prin lipsa

unei autorități centrale, producătorii finali fiind nevoiți să reacționeze la schimbările lanțului reprezentate de etapele de aprovizionare.

Asemănător cu metodele abordate în această lucrare, în 2023, Izdehar M. Aldyaflah și colegii săi [23] abordează opțiunea de a folosi un *smart contract* ca un model de stocare bazat pe etichete. Rezultatele obținute reflectă ca avantaje flexibilitatea contractului și simplitatea, care reduc riscul de introducere a vulnerabilităților, iar ca dezavantaje se numără dependența de Ethereum, fapt care limitează interoperabilitatea și scalabilitatea și numărul mic de operații pe secundă față de o bază de date clasică. Totodată, simplificarea sistemului obligă utilizatorii să efectueze operații suplimentare, cum ar fi criptarea sau extragerea răspunsurilor din rezultatul unei operații. În obținerea acestor rezultate s-a încercat o economisire a costurilor de prelucrare a datelor, deși în lucrarea curentă nu se vor aborda metode de acest tip.

Zhao W. și colegii săi [23] au folosit un sistem de autorizare bazat pe adresa utilizatorului preluată din blockchain, în lucrarea curentă însă s-a folosit o implementare externă în care partea de acces a utilizatorilor este dată de un sistem separat realizat printr-o autentificare prin nume de utilizator și parolă cu scopul de a scădea complexitatea contractului.

Un dezavantaj al acestor contracte este nevoia unei unități de măsură pentru efortul computațional al unei operații în cadrul blockchain-ului, care poartă denumirea de *gas*, de aceea, complexitatea operațiilor și mărimea setului de date sunt limitate la valoarea maximă de prag de *gas* a platformei.

Gas-ul este necesar pentru a executa operații pe blockchain iar acesta este determinat în funcție de diferiți factori precum: complexitatea calculului aritmetice, operațiuni de stocare, lansare (*deploy* – *engl.*) (operație prin care un contract este adăugat pe rețea pentru a fi utilizat), cantitatea de memorie utilizată, utilizarea instrucțiunilor sau a *event emitter*-elor. Utilizatorii sau proprietarii de contracte inteligente plătesc taxe de *gas* pentru a compensa nodurile pentru procesarea tranzacțiilor lor.

Fiecare unitate de *gas* are un preț care reprezintă suma de ETH (ether) pe care utilizatorii sunt dispuși să o plătească pentru *gas*-ul consumat în timpul efectuării unei tranzacții [24]. Prețul este măsurat în diferite unități de măsură, cele mai importante fiind:

- Wei: cea mai mică unitate de măsură unde 1 Wei reprezintă 10^{-18} ETH.
- Gwei: una dintre cele mai utilizate unități de măsură unde 1 Gwei reprezintă 1 Wei sau 10^{-9} ETH.
- ETH: moneda nativă a rețelei Ethereum și unitatea principală în care se efectuează tranzacțiile și plățile comisioanelor de *gas*.

Cantitatea de ETH consumată de o tranzacție în cadrul unui EVM este determinată de un mecanism de piață bazat pe cerere și ofertă, adică, utilizatorul oferă un preț pentru ca tranzacțiile lui să fie incluse rapid în blocuri iar nodurile dedicate minării prioritizează tranzacțiile care oferă cele mai mari recompense. Pentru a preveni situațiile în care doar cererile cu cele mai mari prețuri sunt executate, s-a introdus o taxă de bază ajustată automat în funcție de aglomerația rețelei și un „*bacșiș*” (*priority fee* – *engl.*) pentru a accelera procesarea [25]. Astfel, costul final este calculat cu ajutorul formulei [24]:

$$P = U * (B + F) \quad (1)$$

U = unități de *gas*.

B = prețul de bază al unităților de *gas*, ales în funcție de specificațiile rețelei.

F = „*bacșiș*”, un preț opțional, care crește șansele ca tranzacția să fie procesată mai rapid.

Spre exemplu, pe blockchain-ul Ethereum, o platformă larg utilizată datorită capacității de a executa *smart contracts*, Ethereum Virtual Machine (EVM) execută codul contractelor inteligente, iar fiecare operație care implică modificarea sau procesarea unor date, consumă o cantitate specifică de *gas* spre exemplu, pentru a adăuga un contract, care e una dintre cele mai

costisitoare operații, costul final este aproximativ 0,1 ETH .

Datorită faptului că structura aplicației implică utilizarea unui blockchain împreună cu *smart contract* pentru gestionarea datelor, proiectul poate să fie clasificat ca un *dApp* (aplicație descentralizată), strategie prezentă de puțini ani dar care a luat amploare în domenii variate.

1.1.3. Rețele neuronale

Odată ce stocarea și prezentarea datelor sunt finalizate, s-au folosit rețele neuronale pentru a oferi o analiză asupra unor evenimente simulate prin clasificarea acestora.

Rețelele neuronale sunt inspirate din funcționarea creierului uman, folosite pentru a învăța și a face predicții pe baza unui set de date. Acestea încearcă să reproducă funcționalitatea neuronilor și a sinapselor printr-o serie de noduri care sunt interconectate între ele folosind o rețea de ponderi.

Neuronii rețelei sunt organizați sub formă de straturi care primesc date de intrare, aplică o serie de operații matematice asupra acestora cu ajutorul ponderilor, după care produc o ieșire în urma unei funcții de activare. Pentru a îmbunătăți precizia rezultatului, rețeaua trebuie antrenată pentru a-și modifica ponderile cu ajutorul unui algoritm de optimizare care este aplicat iterativ [26]. Funcția de activare a ultimului strat, adică a celui de ieșire, este cea care determină practic scopul rețelei.

Un aspect important al rețelelor neuronale este capacitatea lor de a generaliza informațiile învățate pentru a face predicții asupra unor date noi, nevăzute anterior. Aceasta se realizează prin antrenarea rețelei pe un set diversificat de date, asigurându-se că nu doar memorează exemplele, ci învață și tiparele esențiale ale acestora

În 2000, Guoqiang Peter Zhang [27] explorează utilizarea rețelelor neuronale în diverse industrii prin clasificare, precum afaceri, știință, industrie și medicină. Rețelele neuronale sunt considerate alternative promițătoare ale abordărilor tradiționale de clasificare, deoarece se adaptează la date și nu necesită specificarea explicită a unui model.

1.2. Tehnologii utilizate

Acest subcapitol explorează tehnologiile utilizate pe parcursul dezvoltării aplicației, analizând modul în care acestea contribuie la funcționarea generală și specifică a aplicației. Se discută despre limbajele de programare, framework-urile, bibliotecile și instrumentele utilizate, evidențiind rolul fiecărei tehnologii în contextul dezvoltării acestei aplicații.

1.2.1. Angular

Angular v0.8.0 [1] este un framework de dezvoltare web open source, creat și menținut de Google, utilizat pentru construirea de aplicații web dinamice și eficiente. Folosit în principal pentru dezvoltarea de aplicații single-page (SPA), Angular permite dezvoltatorilor să creeze interfețe de utilizator bogate și interactive, prin utilizarea unui set de instrumente și biblioteci integrate implicit.

Angular este ideal pentru o platformă de gestiune a datelor datorită mai multor caracteristici esențiale precum:

- Legarea datelor bidirecțională [28]: Aceasta permite sincronizarea automată a datelor între modelul aplicației și interfața utilizator, facilitând gestionarea și actualizarea datelor în timp real, fără necesitatea de cod suplimentar pentru actualizări manuale.
- Arhitectură bazată pe componente: permite dezvoltarea aplicațiilor prin împărțirea

acestora în componente modulare și reutilizabile. Acest lucru duce la un cod mai organizat și mai ușor de întreținut, reducând complexitatea dezvoltării și facilitând extinderea aplicației și implementarea caracteristicilor responsive.

- **Interceptor:** noțiune care ajută la gestionarea cererilor de tip HTTP (HyperText Transfer Protocol) și permite aplicarea de logici suplimentare, cum ar fi gestionarea autorizării sau a erorilor.
- **Componentă de tip „CanActivate”:** Angular dispune de o componentă de tip „CanActivate”, care facilitează separarea tipurilor de utilizatori și protejarea rutelor aplicației, asigurând accesul doar utilizatorilor autorizați la anumite funcționalități sau date sensibile.
- **Comunitate activă:** oferă acces la diferite biblioteci open source ce fac dezvoltarea aplicațiilor mai rapidă și mai ușoară, un exemplu utilizat în această aplicație fiind sweetalert2 [29].
- **Directive structurale [30]:** reprezintă proprietăți ce extind limbajul HTML, permițând adăugarea de funcționalități complexe într-o manieră declarativă.

În cazul de față, Angular este folosit împreună cu SCSS (SASS Cascading Style Sheets) și TS (TypeScript), care este un limbaj de programare superset al JavaScript-ului, ce adaugă suport pentru tipizare statică. Acest lucru este benefic pentru dezvoltatori deoarece permite detectarea erorilor la compilare, îmbunătățind astfel calitatea codului și prevenind problemele comune legate de tipurile de date, cum ar fi erorile de runtime.

În aplicația curentă s-a folosit versiunea 16.2.10.

I.2.2. Python

Python e un limbaj de programare interpretat, de nivel înalt și orientat pe obiect. Este folosit într-o gamă largă de aplicații inclusiv dezvoltare web, analiza datelor și *învățarea automată* (*machine learning* – engl.) care este un subset al inteligenței artificiale. Acesta a fost ales datorită caracteristicii de dinamicitate, sintaxei clare și a flexibilității cu o multitudine de biblioteci dezvoltate de comunitatea creată în jurul limbajului. Mai jos sunt prezentate bibliotecile utilizate:

- **Flask v3.0.0 [31]:** este un micro-framework de dezvoltare web, care se ocupă de *rutarea* (*routing* – engl.) URL (Uniform Resource Locator) și de gestionarea cererilor HTTP, folosit pentru a separa logica de business de restul aplicației și pentru a colabora cu alte biblioteci (json, numpy, os, werkzeug, flask-cors), permițând astfel dezvoltarea de funcționalități diverse.
- **Tensorflow v2.16.1 [32], Keras v3.1.1 [33] :** sunt biblioteci populare de machine learning utilizate în aplicația curentă datorită capabilităților lor de a construi și testa modele de rețele neuronale. TensorFlow oferă o platformă robustă și scalabilă pentru modele complexe, în timp ce Keras simplifică procesul printr-o interfață intuitivă.
- **Web3 v6.15.0 [34]:** este o bibliotecă dedicată interacțiunii cu rețelele descentralizate, oferind instrumentele necesare pentru a realiza comunicarea eficientă cu blockchain-ul, a gestiona elementele rețelei și a apela funcțiile *contractului inteligent*. Împreună cu Web3 este folosit și py-solc-x 2.0.2 [35] care deține logica de compilare a contractelor.
- **SqlAlchemy v2.0.27 [36]:** permite interacțiunea cu bazele de date relaționale cu ajutorul unui ORM (Object-Relational Mapping) care facilitează crearea, modificarea și interogarea schemelor și realizarea operațiilor CRUD (Create, Read, Update, Delete) pe datele tabelor. În acest proiect, SalAlchemy este folosit pentru modelarea unei baze de date SQLite dintr-un fișier local, care stochează informațiile necesare

- utilizatorilor ce au posibilitatea de a se autentifica.
- PyJWT v2.8.0 [37]: permite crearea, semnarea și verificarea token-urilor JSON (JavaScript Object Notation) Web Token (JWT). JWT este un standard deschis [38] care definește un mod securizat de transmitere a informațiilor între două entități. PyJWT oferă funcționalități pentru generarea și validarea token-urilor în conformitate cu specificațiile standardului. Această bibliotecă este utilizată pentru securizarea și gestionarea autorizării utilizatorilor.

1.2.3. Solidity

Solidity v0.8.0 [3], este un limbaj de programare static, inspirat din mai multe limbaje orientate pe obiect, cum ar fi C/C++, JavaScript și Python, special creat pentru dezvoltarea de contracte inteligente pe platforme descentralizate. Prin asemănarea sa cu limbaje familiare, Solidity facilitează procesul de implementare, oferind dezvoltatorilor elemente precum structuri, enumerări, obiecte (contracte) și instrucțiuni de control.

Această componentă a fost aleasă pentru gestionarea seturilor de date care reprezintă informațiile mașinilor stocate prin intermediul contractelor.

1.2.4. Ganache

Ganache v2.7.1 [4], este o aplicație open source care furnizează un mediu de dezvoltare personal, utilizat în special pentru dezvoltarea și testarea de contracte inteligente într-un mediu local, evitând astfel opțiunea de a interacționa cu rețele publice care poate fi contra cost.

Acest instrument oferă o simulare a unui blockchain, împreună cu o serie de funcționalități utile precum o interfață grafică care ajută în verificarea și vizualizarea tranzacțiilor, crearea rapidă de conturi și generarea de blocuri și evenimente. Deși este un mediu de testare, acesta este limitat la nivel de personalizare a blockchain-ului deoarece nu suportă menținerea unei rețele formate din mai multe noduri.

Deoarece Ganache nu suportă menținerea unei rețele formate din mai multe noduri, una dintre modificările posibile la nivelul blockchain-ului este schimbarea permisiunilor între un sistem permis (*permissioned – engl.*) și unul nepermis (*permissionless – engl.*). Alte metode, cum ar fi adoptarea mecanismelor de consens, de guvernare sau validare pe noduri nu pot fi utilizate din cauza acestei limitări. În lucrarea curentă, se utilizează un sistem extern de gestionare a participanților, iar odată ce aceștia au obținut permisiunea, pot aduce orice modificare contractului. Conform analizei lui Paul P. și a colegilor săi [39], acest lucru face ca blockchain-ul utilizat în această aplicație să fie de tip hibrid.

Ganache			
ACCOUNTS BLOCKS TRANSACTIONS CONTRACTS EVENTS LOGS			
CURRENT BLOCK 143 GAS PRICE 2000000000 GAS LIMIT 600000000 HARDFORK MERGE NETWORK ID 5777 RPC SERVER HTTP://127.0.0.1:8545 MINING STATUS AUTOMINING WORKSPACE RECEPTIVE-END SWITCH ⚙️			
MNEMONIC		HD PATH	
six suspect fatal measure lake nuclear essay surround matter join city coin		m/44'/66'/0'/0/account_index	
ADDRESS	BALANCE	TX COUNT	INDEX
0xa188b6f02cbdeF000b4ad42c3285cB949E876a03	992.24 ETH	143	0
0x43621E912746E8C6F82691925C90B5288A3dFf02	1000.00 ETH	0	1
0x66A37e192A79Bf16EB789D0f221C18705673C7f3	1000.00 ETH	0	2
0xd79ce484EaC4985257C9D89466E7Af2BD71110F2	1000.00 ETH	0	3
0x67bff41860cc905C6cf499D7a94c707F06338376	1000.00 ETH	0	4
0xE24287b3c2668986A815fCB869Db1A59c7C8a688	1000.00 ETH	0	5
0x1EDd3d4e004761773138E28AC7d9CDbadD31dF7d	1000.00 ETH	0	6

Figura 1. Pagina de pornire a aplicației Ganache.

I.3. Arhitectura aplicației

Aplicația urmează o structură generală a unui model arhitectural pe *trei niveluri* (*three tier* – engl.) [40]: prezentare, aplicație și date. După cum se poate observa în II.2.4. Anexa 1. , fiecare nivel îndeplinește scopuri distincte, iar comunicarea între acestea se realizează, în general, prin intermediul unor protocoale și canale diferite. Nivelul de prezentare se ocupă de interacțiunea cu utilizatorul, oferindu-i o interfață grafică prin care poate beneficia de funcțiile oferite de platformă. Nivelul de aplicație deține logica de business, care diferă în funcție de cererile platformei. În final, nivelul de date asigură persistența informației, gestionând stocarea, accesul și manipularea datelor. Această separare pe niveluri aduce o îmbunătățire a flexibilității, scalabilității și întreținerii proiectului.

I.3.1. Nivelul de prezentare

Acest nivel se concentrează pe aspectul vizual al aplicației, interacțiunea cu utilizatorul și afișarea datelor. Astfel s-a folosit Angular, pentru a oferi unui posibil client o interfață grafică intuitivă și accesibilă pe majoritatea dispozitivelor. Utilizatorul neautentificat poate vizualiza datele unei mașini dacă introduce o serie de șasiu existentă pe blockchain, iar cel autentificat, pe lângă această funcționalitate, poate aduce noi modificări unei mașini.

A. Flow

Aplicația a fost concepută pentru suportul a trei tipuri de utilizatori: utilizator neautentificat, utilizator autentificat și manager de utilizatori. Utilizatorul neautentificat are acces la pagina principală în fiecare moment, unde poate căuta vehicule sau contacta administratorul pentru posibile colaborări.

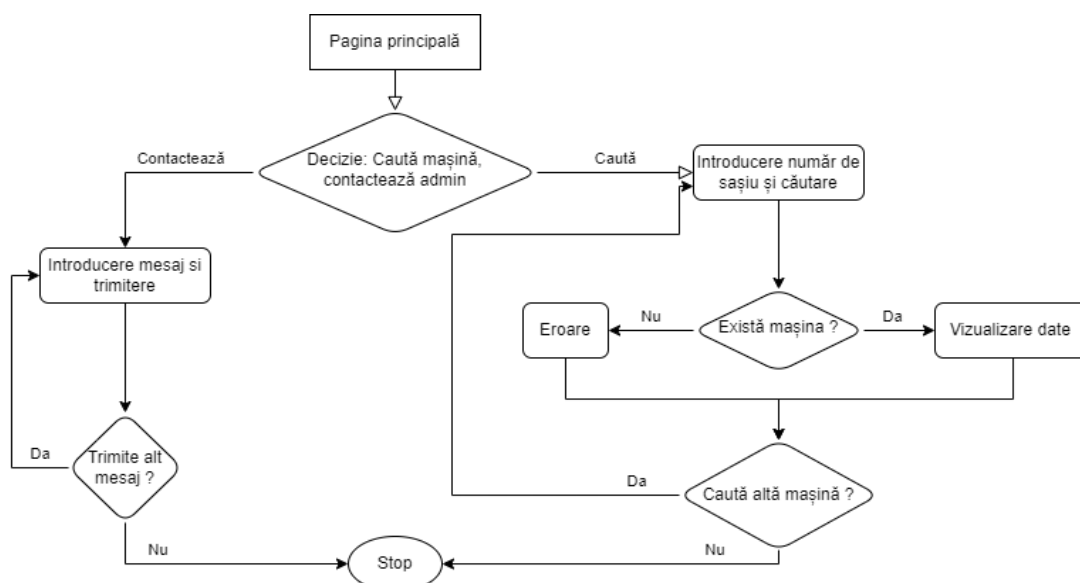


Figura 2. Diagrama de decizie a utilizatorului neautentificat.

Utilizatorul autentificat, pe lângă funcționalitățile disponibile celui neautentificat, beneficiază de accesul la un mecanism de autentificare, facilitând accesul la vehiculele deja înregistrate pe blockchain. Odată ce un vehicul este identificat, utilizatorul autentificat poate contribui la istoricul vehiculului prin adăugarea de modificări sau evenimente relevante.

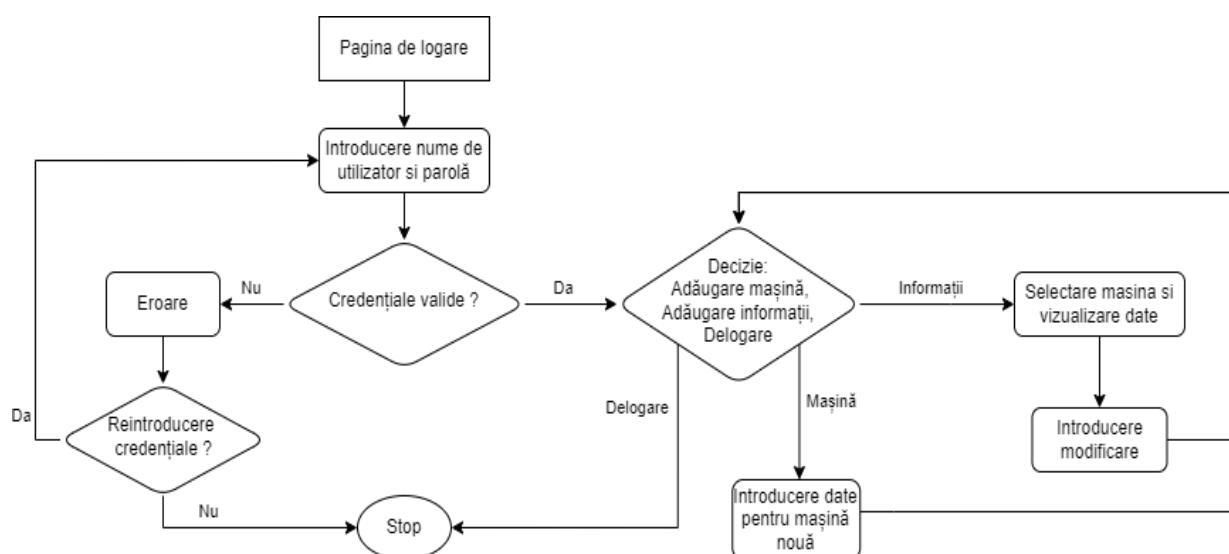


Figura 3. Diagrama de decizie a utilizatorului de tip 2 autentificat.

Cel de-al treilea tip de utilizator are ca scop gestionarea utilizatorilor autentificați. Acțiunile cu cel mai mare nivel de importanță sunt cele de actualizare a stării unui utilizator și de creare a unui cont. Acesta dispune, pe lângă posibilitățile enumerate și de aceleași funcționalități precum un utilizator neautentificat și de mecanismul de conectare în cazul celui autentificat.

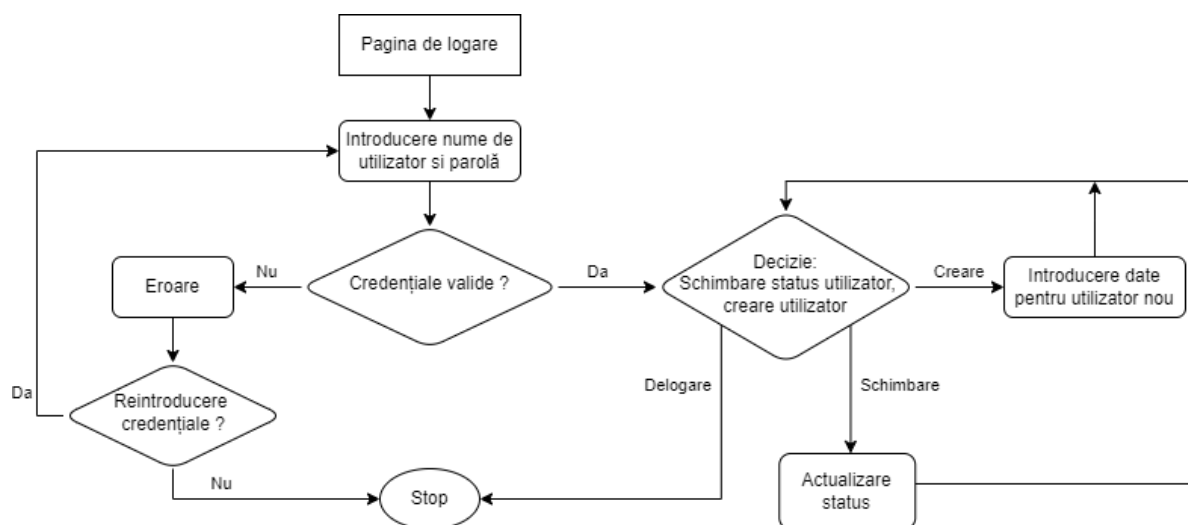


Figura 4. Diagrama de decizie a utilizatorului de tip 3 autentificat.

B. Componente și rute

S-au folosit componente de tip *clasa* (class – engl.) pentru a structura și organiza conținutul fiecărei secțiuni a aplicației, împreună cu elementele de *rută* (Routes – engl.) pentru a gestiona navigarea între aceste secțiuni. Componentele pagină permit crearea unor interfețe de utilizator modulare și reutilizabile, în timp ce rutele facilitează schimbarea dinamică a conținutului afișat în funcție de interacțiunile utilizatorului, asigurând o experiență fluidă și coerentă.

I.3.2. Nivelul de aplicație

Pentru funcționalitatea și logica aplicației, Python împreună cu Flask gestionează legăturile între diverse componente ale aplicației, se ocupă de gestionarea utilizatorilor, implementează măsuri de securitate și facilitează clasificarea datelor.

Pentru a garanta o structură coezivă și funcțională, straturile au fost divizate în module distincte, fiecare cu propriul său scop și obiective bine definite. Astfel, în cazul nivelului *application*, componentele principale sunt: Api (Application Programming Interface) gateway, user, security, RPC (Remote procedure call) client, clasificare. Această metodă facilitează o administrare mai eficientă a resurselor și oferă o flexibilitate crescută, permițând modificări separate în cazul unei schimbări de arhitectură.

A. Module

1) Api gateway

Furnizează un punct de acces unificat pentru comunicarea HTTP între straturile de aplicație și cel de prezentare. Acesta utilizează Flask și bibliotecile asociate pentru a automatiza gestionarea serviciilor web.

Totodată s-a încercat asigurarea unui răspuns corespunzător în diferite situații, precum erori de interogare, folosind o logică bazată pe excepții.

2) User

Administrează operațiile de înregistrare, autentificare și dezactivare a utilizatorilor în detrimentul unui nume de utilizator și a unei parole stocată într-o bază de date în urma aplicării procesului de hashing utilizând algoritmul SHA256 (Secure Hash Algorithm 256-bit).

3) Security:

Pune în practică modelul de autorizare bazat pe JWT (JSON Web Token) pentru a identifica utilizatorii care interacționează cu stratul de prezentare și pentru a restricționa accesul la operațiile de modificare din stratul de date. JWT este compus din trei părți distincte: header, payload și signature:

- Header specifică algoritmul folosit pentru criptare și tipul tokenului (JWT).
- Payload conține informații despre utilizator, sub formă de attribute numite *revendicări* (*claims* – engl.) , inclusiv issuer, subject, expiration time și JWT ID (un identificator generat aleatoriu pentru a asigura unicitatea tokenului), câmpuri preluate din [41].
- Signature este calculată cu ajutorul algoritmului HMACSHA256 (Hashed Message Authentication - SHA256) aplicat pe concatenarea codificată în base64 a câmpurilor header și payload, împreună cu o cheie secretă generată aleatoriu.

4) RPC client:

Se ocupă atât de comunicarea între rețeaua blockchain și server-ul Flask, cât și de preprocesarea datelor schimbate de cele două straturi. Modul de gestionare a informațiilor este reprezentat de funcții care apelează metodele de vizualizare și modificare a datelor oferite de contract.

De asemenea, la nivelul acestui modul au fost implementate funcțiile de lansare (deploy – engl.) și compilare folosind biblioteca Solcx.

5) Clasificare:

S-a folosit Tensorflow pentru a implementa trei modele de clasificare care au la baza rețele neuronale. Două modele iau în considerare anumite date din istoricul mașinii și unul care observă relația dintre anul de fabricație și kilometrii parcurși ai autovehiculului. Datorită utilizării acestei biblioteci, s-a putut realiza o simulare precisă a rețelelor neuronale, configurate independent pentru a satisface cerințele fiecărei clasificări. Structurile finale ale modelelor au fost determinate prin repetate sesiuni de antrenament, variind parametri precum numărul de straturi, numărul de neuroni din fiecare strat și funcția de activare a acestora.

B. Comunicare

1) Comunicare cu nivelul de prezentare

Comunicarea între primele două niveluri este realizată prin cereri HTTP/1.1. Mai exact, clientul trimite cereri prin intermediul browser-ului, în urma unor acțiuni, către server-ul Flask. Acesta, după o serie de procesări, trimite înapoi un răspuns corespunzător datelor primite inițial în cerere. Această comunicare este asigurată de modulul „Api gateway”.

2) Comunicare cu nivelul de date

Comunicarea între stratul de aplicație și stratul de date se desfășoară după cum urmează:

- La nivelul modului de autentificare, se utilizează SQLAlchemy care oferă o interfață de nivel înalt pentru interacțiunea directă cu baza de date.

- Ganache asigură un Remote Procedure Call (RPC) prin care server-ul Flask poate trimite cereri HTTP specificând parametrii și operația dorită. Prin răspunsuri putem afla detalii despre tranzacții precum expeditorul, destinatarul, contractul afectat, cantitatea de gas consumat, etc.

1.3.3. Nivelul de date

Pentru stocarea datelor, s-a integrat Ganache, ales în vederea îndeplinirii scopului de a stoca informații despre autovehicule. În plus, se utilizează o bază de date SQLite locală pentru a gestiona datele utilizatorilor.

Smart contract-ul este dezvoltat cu Solidity și implementează logica de creare, citire și actualizare pentru informațiile unei mașini. Astfel, în gestionarea datelor legate de blockchain, sunt utilizate structuri (struct) pentru a păstra caracteristicile necesare și funcții familiare celor din limbajele orientate pe obiect pentru manipulare.

La acest nivel s-au implementat metode pentru a demonstra imutabilitatea anumitor caracteristici precum numărul de șasiu, producătorul, modelul sau anul de fabricație și transparența acestora cu ajutorul funcțiilor de citire.

Capitolul II. Funcționalitate și implementare

În acest capitol, este descris în amănunt modul de interacțiune al diferitelor tipuri de utilizatori cu interfața grafică a aplicației, precum și modul în care aceasta este implementată. Sunt prezentate diverse scenarii de utilizare, explicându-se cum fiecare tip de utilizator navighează prin interfață, efectuează acțiuni specifice și utilizează funcțiile oferite de aplicație. În ceea ce privește implementarea, capitolul conține secțiuni detaliate cu fragmente de cod explicate, ilustrând cum au fost realizate diferitele componente și funcționalități ale aplicației.

II.1. Funcționalitate

II.1.1. Utilizator neautentificat

Un utilizator neautentificat are posibilitatea de a intra oricând pe platformă, acesta fiind întâmpinat de pagina principală prezentată în Fig. 5.

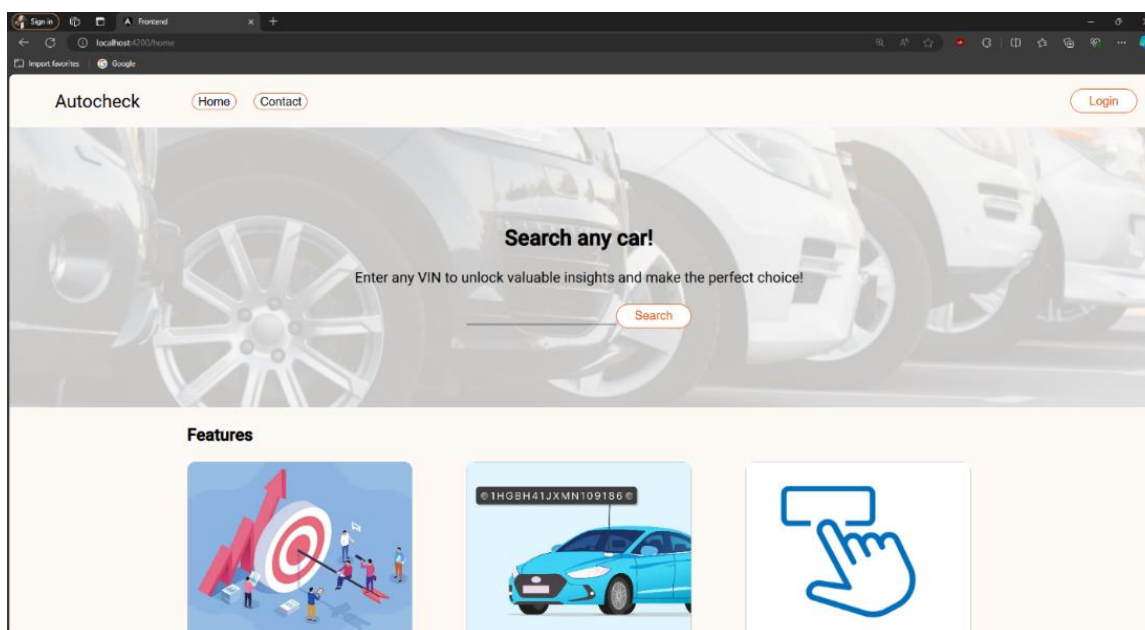


Figura 5. Pagina principală.

Aici poate folosi elementul HTML (HyperText Markup Language) de tip input pentru a introduce o serie de sașiu ce ajută la identificarea mașinii căutate. Butonul „Search” sau apăsarea tastei „Enter” inițiază trimiterea unei cereri HTTP către server-ul de Flask pentru a verifica dacă într-adevăr există un vehicul cu acest număr de identificare, în cazul în care nu există, răspunsul cererii conține un mesaj de eroare prin care interfața grafică afișează un mesaj corespunzător. În caz contrar, dacă seria este găsită, se face o nouă cerere pentru a accesa informația vehiculului, care, odată primită este afișata într-o pagină nouă.

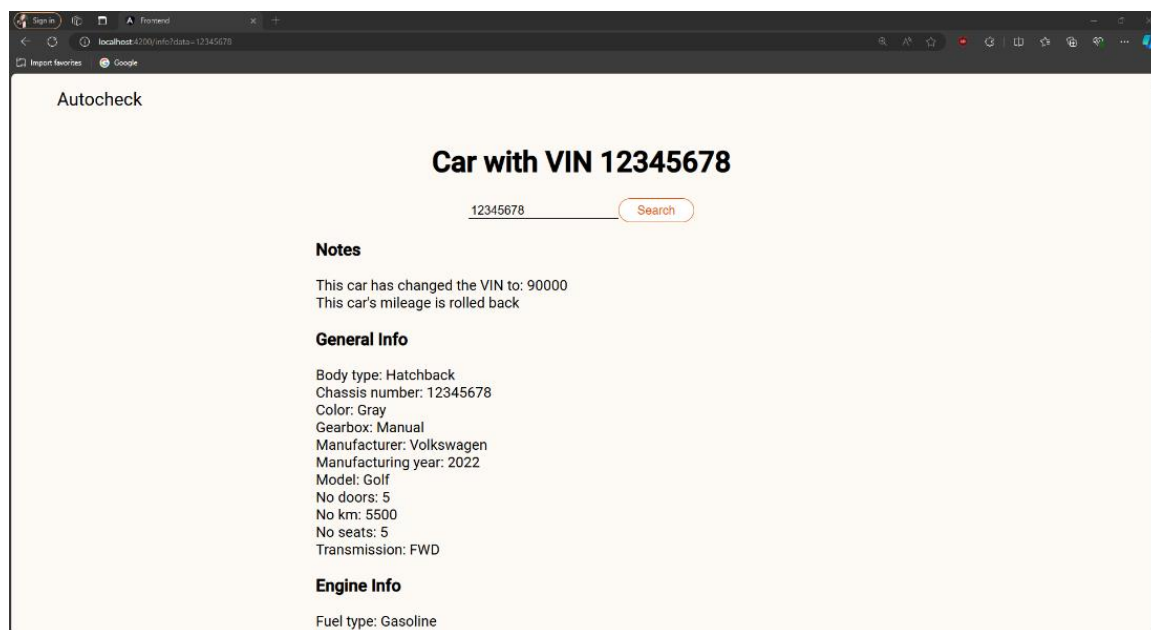


Figura 6. Pagina de informații.

Așa cum se poate observa în Fig. 6 detaliile stocate constau în informații despre starea curentă, liste de modificări, evenimente și modificări suplimentare pentru cazuri speciale, precum și o analiză efectuată asupra acestora.

Revenind la pagina principală, o funcționalitate adițională a acestui tip de utilizator este posibilitatea de a contacta conducerea platformei printr-un scurt formular plasat în partea de jos a paginii. Pentru a finaliza această acțiune, este obligatorie introducerea unui nume și a unei adrese de email, altfel nu se poate iniția acțiunea. Opțional, se poate adăuga și motivul trimerii mesajului, deoarece implicit se deduce că cererea a fost transmisă cu scopul unei colaborări.

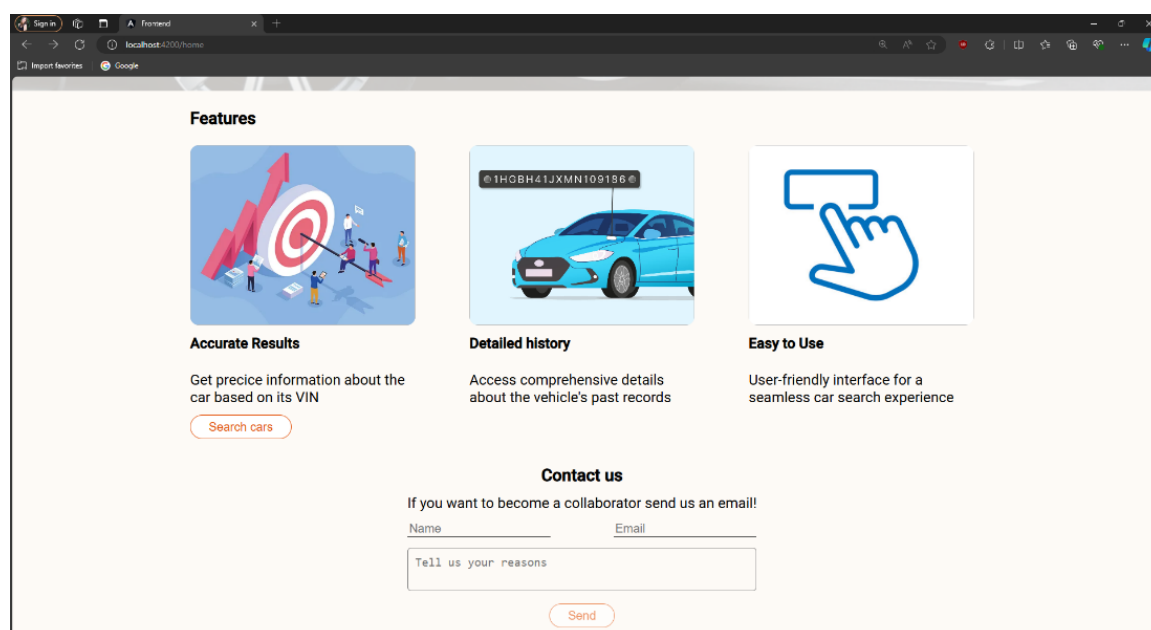


Figura 7. Formular de contact.

II.1.2. Utilizator autentificat

Pe lângă acțiunile pe care le poate realiza un utilizator neautentificat, acesta are și posibilitatea de a se autentifica utilizând numele de utilizator și parola, pe pagina de autentificare accesată prin intermediul butonului „Login”:

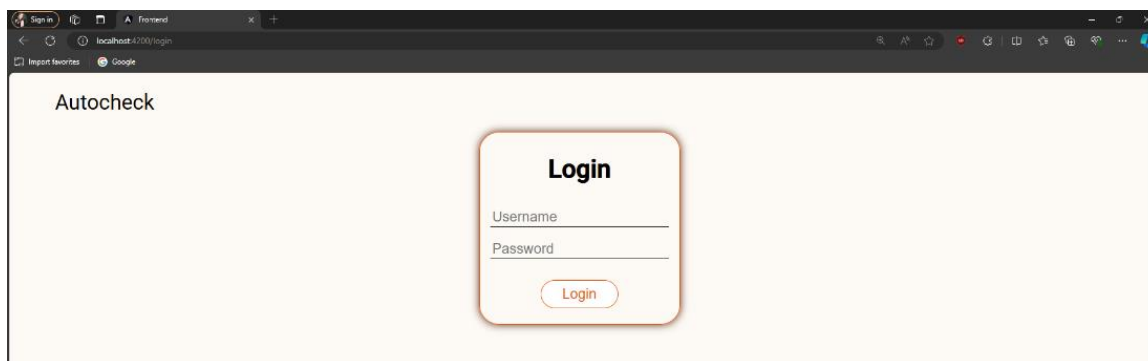


Figura 8. Formular de conectare.

În cazul în care este introdusă una dintre credențiale incorect, este afișat un mesaj cu textul „Wrong Credentials”. Dacă autentificarea are loc cu succes, utilizatorul poate accesa pagina prin care poate gestiona informațiile tuturor mașinilor stocate în blockchain.

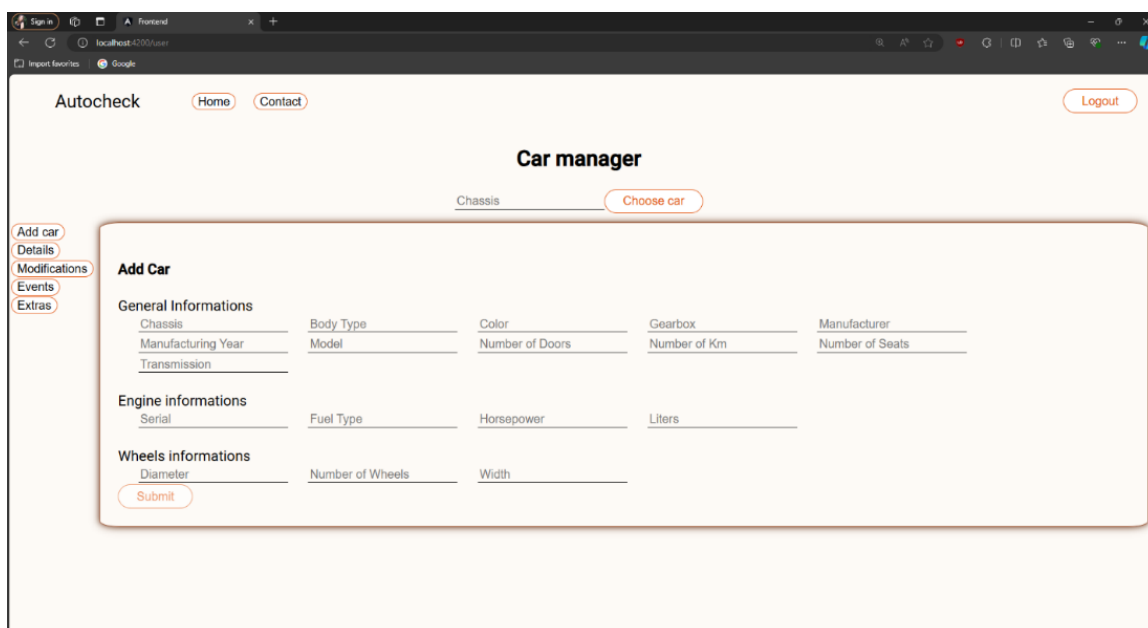


Figura 9. Pagina utilizatorului conectat.

Pe această pagină, elementul de „Search” oferă și o listă a seriilor de șasiu disponibile pentru a facilita căutarea.

Butoanele din partea stângă permit accesul la anumite funcții, astfel:

- „Add car” face să apară componenta cu elementele de input pentru a introduce informațiile unei noi mașini, seria de identificare fiind un câmp obligatoriu, fără de care nu se poate finaliza procesul.

- „Details” afișează informațiile complete, în aceeași manieră ca în Fig. 6.
- „Modifications” oferă mai multe opțiuni directe pentru a actualiza caracteristicile, actualizări personalizate și vizualizarea listei de modificări. Actualizările directe pot fi făcute asupra următoarelor elemente: numărul de kilometri, culoarea, transmisia, caracteristicile motorului, transferul de putere, numărul de locuri, numărul de uși și caracteristicile roților. Modificările personalizate permit utilizatorului să aleagă componenta afectată de schimbare și să adauge o descriere a modificării. Dacă componenta respectivă nu este găsită, există opțiunea de a selecta „Other” și de a descrie în amănunt procesul efectuat.
- „Events” oferă opțiunea de a adăuga și vizualiza istoricul de evenimente. În procesul de adăugare a evenimentelor se alege tipul de eveniment și se adaugă detalii despre acesta, dacă evenimentul nu este prevăzut în elementele disponibile, se poate selecta „Other” și se scrie în amănunt întâmplarea.
- „Extras” oferă posibilitatea de a vizualiza și de a adăuga lista de informații suplimentare. Această funcționalitate este menită să stocheze date cu o importanță minoră, precum accesoriile.

Utilizatorul este autorizat printr-un JWT, având posibilitatea de a rămâne conectat fără a efectua o acțiune până la expirarea token-ului. La finalizarea unei operațiuni, token-ul este *reîmprospătat* (*refreshed* – engl.) în Flask iar datorită acestui mecanism, utilizatorul poate părăsi platforma prin închiderea browserului, rămânând conectat până la expirarea token-ului. Dacă nu dorește acest lucru, se poate deconecta prin butonul „Logout”.

II.1.3. Manager de utilizatori

Un manager de utilizatori, odată ce se autentifică cu numele de utilizator și parola cu succes, este redirecționat către pagina de gestionare a utilizatorilor.

Id	Username	Name	Company	Is Active
1	TestUser	user1	TestSRL	Yes
5	Andrei	andrei	CompanieSrl	Yes
6	Petrescu	petre1	CompanyName	Yes

Figura 10. Pagina de manager de utilizatori.

Aici, managerul poate completa formularul “Create a new user” pentru a adăuga un nou

utilizator autentificat, finalizând procesul prin acționarea butonului “Add User”. De asemenea, poate vizualiza toți utilizatorii înregistrați în tabelul de sub formular. Dacă se apasă o singură dată pe un rând din tabel, acesta este selectat. Dacă se apasă de două ori pe rând, va apărea un dialog-box pentru a confirma actualizarea statusului rândului selectat.

II.2. Implementare

II.2.1. Nivelul de prezentare

S-a implementat o interfață minimală pentru fiecare tip de utilizator folosind HTML, TS și SCSS, adăugând totodată și câteva metode de stilizare care avantajează un aspect responsive prin utilizarea *media queries*, dimensiuni bazate pe procente și aspecte *flexbox*.

Toate declarațiile de componente și importurile de biblioteci sunt scrise în fișierul "app.module.ts". În afară de biblioteca SweetAlert2, s-au folosit doar pachete oferite de framework.

Server-ul Angular este dezvoltat local la adresa URL "http://127.0.0.1:4000/", iar paginile prezentate în subcapitolul II.1. sunt identificate prin rute declarate în fișierul „app_routing.module.ts” cu următoarele căi: „/user”, „/admin”, „/home”, „/info” și „/login”. Când un client se conectează la platformă, acesta este redirecționat către pagina „/home”.

Pentru a facilita reutilizarea codului, în directorul „app/shared” se găsesc clase adnotate cu @Injectable sau @Component. Componentele marcate cu @Component reprezintă de fapt unități compuse din mai multe fișiere, identificate printr-un tag HTML și pot fi utilizate în alte fișiere de acest tip. De exemplu, componenta „render-car” primește ca intrare un JSON ce conține toate datele unei mașini și utilizează mai multe tag-uri precum „render-modifications” și „render-events”, fiind ea însăși folosită în pagini majore precum „info” și „user”.

Fișierele marcate cu @Injectable se bazează pe principiul Dependency Injection, permițând injectarea *serviciului* (service – engl.) în alte clase pentru a beneficia de funcționalitățile acestuia. În acest context, de exemplu, fișierul „http.service.ts” implementează funcții ce conțin logica de trimitere a cererilor către server-ul Flask și este folosit în mai multe părți precum „login” sau „home”. Un exemplu de astfel de cerere, a cărui format este folosit în majoritatea cererilor se poate vedea mai jos.

```
backendURL:string = 'http://localhost:5000';
getCarInfo(chassis:String) {
  return this.http.get<any>(`${this.backendURL}/cars/car?chassis=${chassis}`)
}
```

S-a dezvoltat un *serviciu* de tip HttpInterceptor care monitorizează fiecare răspuns HTTP pentru a salva rolul utilizatorului în *session storage* și token-ul acestuia în *local storage*. Împreună cu *serviciul* de tip CanActivate și obiectele de tip Observable și Subject, este posibilă urmărirea stării și a acțiunilor utilizatorului curent, pentru a-i permite sau interzice accesul la anumite pagini în funcție de nivelul său de autorizare. Astfel, un manager de clienți nu poate modifica o mașină, conform politicilor de acces definite.

Unele dintre cele mai utilizate caracteristici puse la dispoziție de către Angular, în acest nivel, sunt data binding și *directivele* (*directives* – engl.). Data binding este un mecanism prin care se sincronizează datele dintre fișierul TS și cel HTML, facilitând astfel afișarea valorilor variabilelor direct în interfața utilizatorului prin diferite tipuri de legături. Directivele structurale [30], precum *ngIf, controlează afișarea elementelor din DOM (Document Object Model) în

funcție de o condiție specificată. O directivă la fel de importantă este `*ngFor` care e utilizată pentru iterația asupra colecțiilor de date, permițând generarea dinamică a elementelor în template fără a rescrie codul pentru fiecare element în parte.

```
<div *ngIf="showChassisError" class="validation-message-chassis">
  {{ errorMessage }}
</div>
<ng-container *ngFor="let user of users; let index=index">
</ng-container>
```

La nivelul structurii de navigare a aplicației, s-a dorit ca întotdeauna să i se ofere unui utilizator autentificat acces rapid și intuitiv către pagina principală sau către contact în situația în care este necesar să intre în legătură directă cu dezvoltatorii pentru a raporta o *problemă* (bug – engl.) sau pentru a oferi feedback. Această abordare a fost implementată utilizând atributele `routerLink` și funcționalitățile de rutare oferite de framework-ul utilizat.

Utilizarea `routerLink` permite definirea unor link-uri în șabloanele HTML, facilitând astfel navigarea utilizatorului prin aplicație. Împreună cu gestionarea oferită de mecanismul de rutare al aplicației, utilizatorii sunt redirecționați în mod automat către paginile corecte în funcție de acțiunile lor sau de situațiile întâlnite în aplicație.

Această integrare nu doar că optimizează experiența utilizatorului prin asigurarea unei navigări fluide și intuitive, dar contribuie și la eficiența operațională a aplicației prin gestionarea corespunzătoare a rutării și a redirecționărilor. Astfel, utilizatorii au posibilitatea de a accesa rapid informațiile și resursele necesare, contribuind la îmbunătățirea interacțiunii lor cu aplicația și la creșterea satisfacției generale a utilizatorilor.

S-a decis să se implementeze validări ale datelor introduse pentru utilizatorii autentificați folosind verificări precum cele de tip „`Validator.required`”, deoarece aceștia nu au întotdeauna informațiile necesare pentru a determina corectitudinea valorilor introduse. În plus, datele pe care aceștia le introduc sunt permanente în multe cazuri, ceea ce subliniază importanța unei verificări constante.

Împărțirea codului sursă din această aplicație în componente individuale aduce un avantaj major din perspectiva reutilizării. Această practică permite nu doar optimizarea dezvoltării și întreținerii codului, dar și posibilitatea de a folosi aceleași componente în alte aplicații care au nevoie să rezolve aceleași probleme sau să ofere funcționalități similare, cum ar fi verificarea produselor într-un alt context.

Prin structurarea codului în componente, se promovează modularitatea și flexibilitatea, facilitând adaptarea la diverse cerințe și situații de implementare. Această abordare nu doar că sporește eficiența dezvoltării software, dar și contribuie la creșterea scalabilității și a consistenței în implementarea soluțiilor complexe.

II.2.2. Nivelul de aplicație

A. Api gateway

Server-ul Flask este executat local pe portul 5000, având adresa URL "<http://127.0.0.1:5000>". Acesta este format dintr-un fișier principal („`main.py`”) și trei secundare de tip *schema* (*Blueprint* – engl.).

În căutarea unei modalități automate de a verifica cererile înainte de a accesa funcțiile expuse de endpoint-uri și de a gestiona starea server-ului după finalizarea unei cereri, s-a adoptat o abordare bazată pe funcții adnotate cu decoratori specifici.

Această decizie a fost luată pentru a asigura o gestionare eficientă și automatizată a cererilor. Implementarea funcțiilor adnotate cu decoratori a permis o integrare simplificată și consistentă în cadrul aplicației, oferind în același timp un control detaliat asupra verificării cererilor și actualizării stării server-ului.

În fișierul principal al aplicației, se află definită o funcție responsabilă pentru verificarea informațiilor aferente cererilor înainte ca acestea să fie procesate, marcată cu adnotarea `@before-request` și o funcție dedicată modificărilor efectuate înainte de trimiterea răspunsului către client, adnotată cu `@after-request`. În plus, fișierul definește mai multe endpoint-uri esențiale precum: autentificare, deconectare, reîmprospătarea tokenului și contact. Endpoint-urile sunt descrise printr-un șir de caractere care specifică calea unică asociată fiecărei acțiuni expuse de server, împreună cu tipul de cerere HTTP corespunzător.

Datorită politicilor CORS (Cross-Origin Resource Sharing) implementate implicit în Flask, care nu permit cereri între domenii diferite, a fost necesară dezactivarea acestei restricții pentru a permite comunicarea între server-ele din domenii diferite. Această operațiune a fost realizată utilizând funcția `CORS(blueprint)` din biblioteca Flask-cors.

Schema din fișierul „cars_blueprint.py” conține endpoint-uri care furnizează informații detaliate despre vehicule și oferă posibilitatea de a verifica dacă un anumit număr de șasiu există pe blockchain. În subcapitolul II.2.1. este prezentată o astfel de operație care furnizează detalii despre un autovehicul în funcție de numărul de șasiu printr-o cerere de tip GET către adresa URL „http://localhost:5000/car?chassis=”, endpoint care aparține *schemei* „cars_bp”.

```
cars_bp = Blueprint('cars', __name__)
CORS(cars_bp)
@cars_bp.route('/car', methods=['GET'])
def get_all_car():
    chassis = request.args.get('chassis')
```

Endpoint-urile din *schema* „registry_blueprint.py” sunt restricționate astfel încât să fie accesibile doar utilizatorilor cu rolul „USER”. Asta însemnând că doar utilizatorii autentificați cu acest rol specific pot accesa și beneficia de funcționalitățile expuse de aceste endpoint-uri. În schimb, endpoint-urile definite în *schema* „users_blueprint.py” sunt disponibile exclusiv pentru utilizatorii cu rolul „ADMIN”. Această restricție de acces este implementată prin verificarea ID-ului stocat în câmpul “sub” asociat token-ului din cerere utilizând funcția decorată cu `@before_request` din fișierul principal.

Împărțirea acestui modul în fișiere care conțin obiecte de tip „Blueprint” reprezintă un avantaj, deoarece subliniază posibilitatea de a dezvolta server-ul cu o diversitate de endpoint-uri care furnizează noi funcționalități benefice pentru clienți sau pentru dezvoltatorii interfeței grafice.

Această abordare permite o organizare clară și modulară a funcționalităților, facilitând extinderea și gestionarea acestora în cadrul aplicației. Prin implementarea acestei structuri, se optimizează gestionarea codului și se promovează o scalabilitate eficientă a server-ului, îmbunătățind în final experiența utilizatorilor și oferind mai multă flexibilitate în implementarea de noi caracteristici și servicii.

Funcția decorată cu `@errorhandler(Exception)` asigură gestionarea centralizată a erorilor, returnând mesajele și codurile de eroare corespunzătoare. În cazul în care se face o cerere către un endpoint neimplementat, se returnează codul 404 împreună cu mesajul „Endpoint Not Found”. Pentru a gestiona diferitele cazuri din logica de business, s-a implementat o clasă personalizată care moștenește clasa `Exception` denumită “CustomException”. Aceasta permite stocarea mesajului și a codului de eroare, în funcție de erorile întâlnite pe parcursul implementării care sunt afișate în momentul în care excepția ajunge la metoda `handle_custom_exception()`.


```
@app.errorhandler(Exception)
def handle_custom_exception(error):
    if isinstance(error, CustomException):
        return jsonify({'error': error.get_message()}),
    error.get_status_code()
    if isinstance(error, werkzeug.exceptions.NotFound):
        return jsonify({'error': "Endpoint Not Found"}), 404
    else:
        return jsonify({'error': error.args[0]["message"]}), 500
```

Spre exemplu, pentru a gestiona situațiile de conflict din fișierul „database_repo.py”, cum ar fi încercarea de a adăuga un utilizator cu un nume deja folosit, se poate *arunca* (*raise* – engl.) o excepție de tip `CustomException` cu un mesaj intuitiv împreună cu un cod de eroare corespunzător (de exemplu, 409). Această abordare permite returnarea unui răspuns adecvat către client, menținând în același timp logica de eroare centralizată.

```
def add_user(username, name, password, company, role):
    user = session.query(User).filter_by(username=username).first()
    if user:
        raise CustomException("User with this username already exists!", 409)
```

B. User

Acest modul este reprezentat de fișierele „user_service.py” și „database_repo.py”, care înglobează operațiile de înregistrare, autentificare și dezactivare a utilizatorilor. Funcțiile din primul fișier gestionează autentificarea utilizatorilor și determină rolul acestora, în timp ce în cel de-al doilea fișier sunt implementate următoarele:

- Clasă ce descrie tabela utilizatorilor din II.2.4. Anexa 2.
- Sesiunea de conexiune cu fișierul din același director „repo/users.db”

```
file_path = 'repo/users.db'
engine = create_engine(f'sqlite:/// {file_path}')
Base = declarative_base()
Base.metadata.create_all(engine)
Session = sessionmaker(bind=engine)
session = Session()
```

- Funcțiile CRUD, care finalizează sfârșitul modificărilor printr-un commit ce semnifică salvarea acestora.

```
def deactivate_user_by_id(id):
    user = get_user_by_id(id)
    user.is_active = False
    session.commit()
```

Această implementare a informațiilor unui utilizator permite extinderea în orice moment a tabelului actual cu proprietăți importante care identifică utilizatorii sau cu tabele de legătură ce pot menține informații despre acesta sau despre compania din care face parte.

În plus, baza de date actuală este una minimală, utilizată în principal pentru scopuri de testare. Structura sa permite o extindere facilă, inclusiv prin adăugarea unui server dedicat care să administreze o bază de date precum Docker sau alte soluții ce facilitează separarea serviciilor sau

introducerea opțiunilor de descentralizare. Printre aceste opțiuni se numără și dezvoltarea unui contract care să stocheze aceste informații și să gestioneze atât datele utilizatorului, cât și permisiunile acestuia asupra unui contract.

Cu toate acestea, conform detaliilor menționate în primul capitol, aceste măsuri nu au fost adoptate din cauza necesității de a menține o complexitate redusă, favorizând astfel un mediu de testare simplificat și eficient.

Această abordare nu numai că facilitează adaptabilitatea și scalabilitatea sistemului, dar și îmbunătățește capacitatea de gestionare a datelor și securitatea, oferind posibilități extinse pentru dezvoltarea și optimizarea aplicațiilor viitoare.

C. Security

Modulul „security” este reprezentat de directorul cu același nume și are funcționalități implementate pentru verificarea și gestionarea token-urilor, precum și pentru gestionarea constantelor care nu ar trebui să apară în codul sursă.

Fișierul „secrets-reader.py” este responsabil pentru gestionarea constantelor secrete stocate într-un fișier local „secrets.txt”. Acesta conține două funcții principale: una pentru actualizarea secretelor și alta pentru citirea acestora pe baza unei chei furnizate. În cazul în care cheia nu este găsită, se va *arunca* o excepție. În plus, fișierul definește clasa „Keys” conform Anexei II.2.4. Anexa 2. , care moștenește clasa `Enum` pentru a facilita enumerarea cheilor posibile.

Fișierul „security-service.py” este responsabil pentru efectuarea verificărilor pe datele utilizatorilor și pentru gestionarea token-urilor, incluzând generarea, reîmprospătarea și verificarea acestora.

Procesul de creare a unui token constă în următorii pași:

1. Atribuirea *claims*

- *Payload* este setat la „http://localhost:5000” .
- *Subject* este ID-ul utilizatorului care va primi token-ul.
- *Expiration time* este calculat folosind timpul curent adunat cu timpul în minute citit din fișierul „secrets.txt”.
- JWT ID este un identificator generat aleatoriu cu biblioteca „uuid” pentru a asigura unicitatea token-ului.

2. Utilizând funcția `jwt.encode()`, se generează token-ul final în formatul „Header.Payload.Signature”, utilizând algoritmul de criptare HMACSHA256. În acest proces, se folosește o cheie secretă generată aleatoriu, formată dintr-o combinație de caractere diverse. Această cheie este stocată în fișierul „secrets.txt”. Token-ul final integrează, de asemenea, informațiile de tip *claims* definite în etapa anterioară.

Reîmprospătarea token-ului implică mai multe etape tehnice. În primul rând, se decodează JWT-ul primit și se verifică *revendicările*. După ce această verificare este realizată cu succes, se generează un nou token cu aceleași *revendicări*, însă cu timpul de expirare actualizat.

Fișierul „token-service.py” are rolul de a gestiona operațiunile de scriere și citire a token-urilor invalidate într-un fișier local denumit „blacklist.txt”, cu scopul de a monitoriza JWT-urile care au fost deja utilizate sau prezintă erori.

D. RPC client

Acest modul este reprezentat în întregime de fișierul „CarRegistry.py”. Acesta conține funcții pentru conversia datelor între formate JSON și tuple, și invers, precum și o clasă numită „CarRegistry” care gestionează toate aspectele legate de conexiunea, compilarea, administrarea datelor și dezvoltare.

Inițial, a fost decisă implementarea server-ului care să gestioneze logica aplicației folosind framework-ul Spring [42] și limbajul de programare Java [43]. Planul părea promițător pentru a asigura o integrare robustă și pentru a facilita comunicarea cu contractele Solidity prin intermediul bibliotecii Web3j [44]. Însă, în timpul dezvoltării, s-au întâmpinat dificultăți semnificative rezultate în urma compilării contractului Solidity. Problemele au apărut în special în compatibilitatea tipurilor de date între clasa generată automat pentru apelarea funcțiilor din fișierul "Car.sol" și tipurile de date specifice limbajului Java.

După stabilirea cu succes a conexiunii cu serverul RPC, au continuat să apară erori frecvente în momentul apelării anumitor metode din clasa rezultată. Aceste erori au complicat integrarea și au afectat funcționarea corectă a aplicației. Astfel, în ciuda eforturilor inițiale de a utiliza Spring pentru implementare, a fost necesar să se reevalueze opțiunile pentru a asigura o integrare și o funcționare adecvată în cadrul aplicației bazate pe tehnologii blockchain.

Deoarece problema a persistat și nu s-a găsit o soluție elegantă pentru rezolvare, s-a decis să se încerce compilarea și conexiunea cu serverul RPC folosind un fișier JavaScript. Însă, soluția identificată s-a dovedit a fi depășită și au apărut erori la compatibilitatea versiunilor atunci când acest fișier a fost integrat în serverul Angular. În mod teoretic, acest lucru ar fi trebuit să funcționeze corect, deoarece TypeScript este un superset al JavaScript și ar trebui să fie capabil să utilizeze orice cod JavaScript fără probleme de compatibilitate.

Pentru a depăși această situație, s-a optat pentru utilizarea unui fișier TypeScript, ceea ce a dus la o conexiune reușită. Cu toate acestea, chiar și cu această soluție, au apărut noi provocări legate de tehnologiile implicate.

Datorită necesității de a utiliza mai multe valori constante pentru a configura conexiunea și pentru a efectua modificări esențiale pe server, a devenit evident că aceste valori nu ar trebui să fie disponibile publicului larg. Astfel, s-a luat decizia de a implementa un server dedicat care să gestioneze aceste operațiuni, inclusiv compilarea și conexiunea necesară.

În acest scop, s-a ales să se utilizeze Python, împreună cu biblioteca Web3, pentru a facilita o conexiune fluentă cu serverul RPC. Această abordare a demonstrat eficiență în comparație cu alte tehnologii anterioare, unde nu s-a reușit cu succes implementarea funcționalităților de deploy. Soluția a implicat o integrare manuală reușită datorită flexibilității și capacității de adaptare a limbajului Python în gestionarea acestor operațiuni critice.

În plus, până la implementarea acestui server separat, procesul de compilare, deploy și testare se baza pe platforma Remix [45], ceea ce adesea îngreuna dezvoltarea contractului și adăugarea de funcționalități noi. Alegerea limbajului Python a permis o mai mare libertate și control asupra procesului de dezvoltare și a adus beneficii semnificative în eficiența și fiabilitatea aplicației blockchain.

Conexiunea cu serverul de blockchain se realizează prin RPC la adresa URL "http://127.0.0.1:8545" iar cu ajutorul bibliotecii Web3 este facilitată această interacțiune.

Deși există diverse tehnologii open source disponibile și destinate dezvoltării și compilării fișierelor Solidity prin simpla apăsare a unor butoane sau executarea unor comenzi în medii de dezvoltare precum VS Code (Visual Studio Code) [46] sau Truffle Suite [47], în proiectul de față s-a optat pentru utilizarea PyCharm [48] în etapa de dezvoltare, pentru a evita alternarea între diferite unelte. În acest sens, am utilizat o extensie dedicată limbajului Solidity [49] în PyCharm, care aduce beneficii semnificative în implementarea contractelor, însă nu contribuie la procesul

lor de compilare.

În acest caz s-a implementat un sistem automat de recompilare bazat pe hash-urile fișierelor. Aceste hash-uri sunt stocate în „secrets.txt”. În momentul în care clasa `CarRegistry` este instanțiată, hash-urile curente ale contractelor sunt comparate cu cele din „secrets.txt”. Dacă se detectează o diferență, sistemul lansează un *subprocess* pentru a recompila contractele, utilizând biblioteca `Solcx`.

```
solcjs = secrets.get_secret(secrets.Keys.SOLCX_PATH)
command = [solcjs, "--optimize", "--abi", "--bin", "-o", ".",
"CarRegistry.sol"]
result = subprocess.run(command)
```

Acest mecanism servește și ca parte a procesului de dezvoltare, deoarece în urma compilării sunt generate două fișiere noi: unul de tip ABI (Application Binary Interface) și unul de tip BIN (Binary), necesare pentru accesarea funcțiilor contractului stocat pe rețea. Dacă aceste fișiere sunt diferite, conexiunea nu poate fi stabilită, motiv pentru care este necesară o acțiune de *deploy* a noului contract la o altă adresă în blockchain. De asemenea, în cazul în care apare o excepție în momentul încercării de a obține obiectul care face conexiunea cu contractul de pe rețea, acesta va fi *lansat* la o altă adresă.

Dat fiind faptul că autentificarea se efectuează prin intermediul unui nume de utilizator și parolă, operațiile directe pe blockchain sunt realizate folosind un singur cont (reprezentat de o adresă) din cele 10 oferite implicit, celelalte 9 fiind rezerve pentru cazurile în care nu este disponibil suficient *gas* pe adresa utilizată. Fiecare cont are alocat un sold de 100 Ether (ETH) la instanțierea server-ului, însă, întrucât acesta este un mediu simulat, această valoare poate fi modificată oricând în scopuri de testare. Într-un scenariu real, celelalte conturi ar trebui să se ocupe de activități de *minare* (*mining* - engl.) (validare a tranzacțiilor) pentru a asigura disponibilitatea de *gas* necesar.

Odată ce conexiunea este realizată cu succes, putem accesa funcțiile oferite de smart contract prin realizarea unor tranzacții. Pentru a iniția o tranzacție care alterează starea rețelei, trebuie identificat utilizatorul care face asta. Astfel mai întâi este construit un JSON care conține adresa contului care face tranzacția (în acest caz, adresa contului implicit), prețul *gas*-ului setat la 20 Gwei (valoare implicită la crearea serverului), și un *nonce* (Number Used Only Once) care reprezintă numărul total de blocuri din rețea.

```
def get_transaction_object(self):
    nonce = self.web3.eth.get_transaction_count(self.default_account)
    return { 'from': self.default_account,
            'gasPrice': self.web3.eth.gas_price,
            'nonce': nonce,
            }
```

Apoi, acest JSON este atribuit funcției pe care dorim să o executăm, rezultând un obiect de tip „TxParams”.

```
transaction = self.contract.functions.addChassis(userid,
chassis).build_transaction(self.get_transaction_object())
self.sign_and_send_transaction(transaction)
```

Acest obiect trebuie semnat cu cheia privată a contului care a inițiat tranzacția, aceasta fiind stocată în „secrets.txt”.

```
def sign_and_send_transaction(self, transaction):
    signed_transaction = self.web3.eth.account.sign_transaction(
transaction,
self.PRIVATE_KEY)
    transaction_hash =
self.web3.eth.send_raw_transaction(signed_transaction.rawTransaction)
    return transaction_hash, transaction_hash.hex()
```

Dacă tranzacția are succes, această funcție va returna o adresă prin care putem verifica detaliile tranzacției prin metoda `web3.eth.wait-for-transaction-receipt(hash)`, cum ar fi expeditorul, destinatarul, contractul afectat, cantitatea de gas consumat, etc. Procesul descris anterior poate fi vizualizat în II.2.4. Anexa 3.

Dacă funcția dorită este una de *vizualizare* (*view* - engl.) a datelor și nu face modificări pe rețea, poate fi apelată fără a necesita o semnătură.

În cazul în care apelăm o funcție care necesită un număr mare de parametri, este crucial să ne asigurăm că numărul și tipul parametrilor sunt corecte. În caz contrar, Ganache va genera o excepție. De aceea, înainte de a crea cererea, se face o conversie din JSON în tuplu (Fig. 11). După primirea unui răspuns, informația este prezentată sub formă de tuplu, motiv pentru care este necesară o serializare în format JSON pentru a ușura citirea și accesarea datelor.

```
{
  "carInfo": {
    "chassisNumber": "1ABC2345",
    "manufacturingYear": 2019,
    "manufacturer": "Toyota",
    "model": "Camry",
    "bodyType": "Sedan",
    "gearbox": "Automatic",
    "color": "Silver",
    "noSeats": 5,
    "noDoors": 4,
    "noKm": 25543,
    "transmission": "FWD"
  },
  "engineInfo": {
    "serial": "ABC123",
    "liters": "2.5",
    "horsepower": 203,
    "fuelType": "Gasoline"
  },
  "wheelsInfo": {
    "noWheels": 4,
    "diameter": 17,
    "width": 7
  }
}
```

(('1ABC2345', 2019, 'Toyota', 'Camry', 'Sedan', 'Automatic', 'Silver', 5, 4, 25000, 'FWD')
('ABC123', '2.5', 203, 'Gasoline')
(4, 17, 7))

Figura 11. Date sub formă de JSON (stânga) și tuplu (dreapta).

Pe lângă aceste funcții, la nivelul acestei clase s-au implementat și apeluri ale metodelor de vizualizare și modificare a unui atribut din smart contract. De asemenea, a fost implementată o funcție care determină dacă kilometrii au fost dați înapoi pe baza istoricului, precum și o funcție care returnează numărul mediu de accidente, daune, vânzări, reparații și mentenanțe realizate pe an.

E. Clasificare

Modulul acesta implementează trei modele de clasificare care au la bază rețele neuronale și este conturat de directorul „ml”. Două modele iau în considerare anumite date din istoricul mașinii și unul care observă relația dintre anul de fabricație și kilometrii parcurși ai autovehiculului. Datorită utilizării bibliotecii Tensorflow, s-a putut realiza o simulare precisă a rețelelor neuronale, configurate independent pentru a satisface cerințele fiecărei clasificări. Structurile finale ale modelelor au fost determinate prin repetate sesiuni de antrenament, variind parametri precum numărul de straturi, numărul de neuroni din fiecare strat și funcția de activare a acestora.

Aceste modele sunt reprezentate de trei clase: ModelCD, ModelYK și ModelMS, care moștenesc o clasă abstractă „Model” și care sunt plasate în fișierele cu aceleași nume. Clasa părinte definește modelul de structurare a acestora. Astfel, fiecare clasă copil trebuie să aibă un fișier pentru ponderile finale ale rețelei, un fișier pentru ponderile de antrenare, precum și două fișiere pentru *trăsăturile* (*features* – engl.) și *clasele* (*labels* -engl.) corespunzătoare setului de date de antrenare.

În plus, fiecare dintre aceste clase trebuie să aibă 4 metode esențiale:

- Train: în care se antrenează modelul.
- Generate_data: în care sunt generate și returnate trăsăturile și clasele setului de date conform regulilor modelului.
- Load: care încarcă ponderile finale rezultate în urma antrenării.
- Predict: care primește un singur set format din trăsături și clase și returnează valoarea aproximată de rețea.

În procesul de antrenare, deși s-au observat performanțe similare cu SGD (Stochastic Gradient Descent), a fost adoptat algoritmul Adam, recunoscut în domeniul optimizării datorită capacității sale de a ajusta individual rata de învățare pentru fiecare parametru, în vederea minimizării pierderilor într-un mod eficient și rapid.

Având în vedere că obiectivul rezultatului dorit este de natură binară, s-a optat pentru utilizarea funcției de pierdere Binary Cross Entropy [50] prezentată în formula (2), recomandată în special pentru astfel de situații, unde există două clase distincte: clasă pozitivă și clasă negativă. Funcția calculează discrepanța dintre probabilitățile prezise de model și etichetele reale ale datelor de antrenament, furnizând astfel o metrică a diferenței dintre distribuția de probabilitate estimată și distribuția reală a etichetelor. Scopul constă în minimizarea acestei pierderi, astfel încât modelul să devină mai precis în clasificarea datelor și să generalizeze mai bine pentru seturile noi de date.

$$loss = -\frac{1}{N} \times \sum_{i=1}^N y_i \times \log \log (P(y_i)) + (1 - y_i) \times \log (1 - P(y_i)) \quad (2)$$

În straturile ascunse ale rețelei, funcția de activare adoptată este Rectified Linear Unit (ReLU) [51] datorită performanței și a capacității de a menține valorile pozitive intacte după cum se poate observa în formula (3). De asemenea, funcția ReLU contribuie la accelerarea convergenței algoritmului de antrenare și previne dispariția gradientilor în timpul procesului de optimizare.

$$f(x) = \max (0, x) \quad (3)$$

Pentru stratul de ieșire al rețelei, se utilizează funcția sigmoidă [52] descrisă de formula (4), deoarece aceasta convertește pentru fiecare parametru de ieșire din rezultat într-o probabilitate

care aparține intervalului (0,1), ceea ce este adecvat în contextul unei probleme de clasificare binară unde un set de date poate prezenta apartenența la mai multe clase.

$$f(x) = \frac{1}{1 + e^{-x}} \#(4)$$

În scopul asigurării generalizării, s-au configurat 100 de epoci și o dimensiune a *loturilor* (*batch* – engl.) de 32, pentru procesul de antrenare, unde actualizarea ponderilor are loc după trecerea fiecărui lot prin rețea. S-a ales această dimensiune a loturilor deoarece valorile mai mari ar fi putut cauza probleme legate de memorie, *supraantrenare* (*overfitting* – engl.) și viteza de antrenare. Pe de altă parte, valorile prea mici ar fi dus la o viteză de antrenare mai redusă sau ar fi crescut riscul de *overfitting*.

Datele folosite în procesul de antrenare și validare sunt create printr-un proces de generare aleatoriu, care respectă anumite reguli stabilite anterior. Aceste reguli variază în funcție de obiectivele fiecărei rețele neuronale. În final s-a propus o cantitate de 2000 de seturi de date pentru fiecare model, a căror clase sunt setate inițial în funcție de regulile sale.

Pentru a putea alege arhitectura finală a fiecărei rețele, s-a efectuat o selecție a datelor de intrare, alocând 80% pentru procesul de antrenare și 20% pentru validare. În cadrul evaluării performanței structurii alese, metricile monitorizate includ eroarea medie pătratică (MSE) și acuratețea. Platforma TensorFlow oferă implicit opțiunea de a observa aceste metrici în timpul antrenării pentru lotul curent. Totuși, prin setarea unui parametru „validation-split”, se poate observa performanța modelului pe un procent din datele de antrenare în timpul procesului. Aceste evaluări nu influențează ponderile modelului și nu afectează procesul de antrenare propriu-zis. Astfel, la fiecare iterație, se poate monitoriza acuratețea, MSE și *pierdere* (*loss* – engl.) atât pe datele de antrenare, cât și pe cele de validare.

```
self.model.compile(optimizer='adam', loss="binary_crossentropy",  
metrics=['accuracy', 'mse'])  
self.model.fit(train_features, train_labels, epochs=100,  
batch_size=32, validation_split=0.2)
```

II.2.3. Smart contract

Acest modul este definit de două fișiere de tip .SOL (Solidity): "Car.sol" și "CarRegistry.sol". Așa cum s-a precizat în secțiunea I.3.3., aceste fișiere integrează logica de stocare și de preluare a informațiilor autovehiculelor. Primul fișier, „Car.sol”, se ocupă de organizarea datelor referitoare strict la o mașină, în timp ce al doilea fișier, „CarRegistry.sol”, utilizează instanțe ale contractului de tip *Car* pentru a construi un registru întreg de autovehicule.

În fișierul “Car.sol” se poate observa că datele au fost împărțite în structuri ce îndeplinesc anumite funcții specifice. Astfel, caracteristicile generale, cele despre motor și cele despre roți sunt încapsulate în structurile „GeneralInfo”, „EngineInfo” și „WheelsInfo”, conform în II.2.4. Anexa 3. Pentru aceste seturi de date, au fost implementate funcții de tip view, destinate recuperării valorilor, astfel încât să nu efectueze alte operații suplimentare, ci doar să returneze datele.

```
function getCarInfo() public view returns (GeneralInfo memory) {  
    return currentCarInfo;  
}
```


Pentru atributele fiecărei structuri s-au implementat funcții de actualizare. Pe lângă parametrul destinat modificării, aceste funcții necesită și un timestamp care reprezintă momentul în care s-a efectuat modificarea, precum și un ID al utilizatorului care a realizat-o.

```
function modifyColor(uint256 timestamp,uint64 _userId, string memory _color)
public {
    addModification(timestamp,_userId, Part.Exterior ,
string(abi.encodePacked("Color changed from ",currentCarInfo.color, " to ",
_color)));
    currentCarInfo.color=_color;
}
```

Unele date însă, cum ar fi numărul de șasiu, modelul și numărul de fabricație, nu au implementate funcții de actualizare fapt care asigură imutabilitatea acestora, în timp ce altele, cum ar fi dimensiunile roților, culoarea și numărul de kilometri, se pot modifica prin intermediul metodelor gândite cu acest scop.

În cadrul unei tranzacții, este prezent conceptul de *event emitter*, care evidențiază datele încărcate în blockchain în timpul acesteia. Totuși, pentru a căuta o anumită schimbare, ar fi necesar să se filtreze toate tranzacțiile, ceea ce ar conduce la un timp considerabil de procesare. Prin urmare, s-a optat pentru o metodă mai eficientă de urmărire a schimbărilor, și anume, salvarea timestamp-ului și a ID-ului utilizatorului la fiecare modificare sau eveniment. Această abordare permite stocarea unei cantități mai mari de date pentru o mașină, având ca beneficiu o căutare rapidă.

La fiecare actualizare, informațiile tranzacției sunt adăugate la sfârșitul unui vector de structuri de tip „Modification”. Aceste structuri conțin detalii despre modificare, cine a efectuat-o, tipul modificării și momentul în care a fost realizată. Modificările generale pot fi adăugate și retroactiv, dar dacă sunt făcute în prezent, timestamp-ul trebuie să fie egal cu valoarea 0. Această logică este aplicată și structurilor de tip „Extra” și „Event”, împreună cu tipurile evenimentelor, prezentate în Anexa II.2.4. Anexa 3.

```
struct Modification {
    uint256 timestamp;
    uint64 userId;
    Part enumPart;
    string details;
}
```

În plus, clasa conține și un atribut denumit „transfer”, de tip string. Acest atribut a fost adăugat cu scopul de a informa utilizatorul că seria de șasiu a fost modificată. În cazul în care acest atribut este prezent, utilizatorul ar trebui să efectueze următoarele modificări asupra obiectului care are seria de șasiu indicată de valoarea acestui atribut.

Contractul CarRegistry conține două dicționare, un vector de string-uri și un eveniment. Evenimentul este utilizat în funcția `constructor()` pentru a confirma lansarea clasei. Vectorul de string-uri are rolul de a menține toate șasiurile stocate, iar dicționarul „carExists” folosește șasiul drept cheie și o valoare booleană ca valoare, fiind utilizat pentru o verificare rapidă a vehiculelor prezente. Proprietatea „cars”, de tip „mapping”, este și ea un dicționar, cheia fiind un string ce reprezintă șasiul și ca valoare o instanță a contractului Car, permițând astfel accesul facil la funcțiile obiectului.


```
function getCarInfo(string memory _chassis) public view returns (GeneralInfo memory) {
    require(carExists[_chassis], "Car does not exist");
    return cars[_chassis].getCarInfo();
}
```

În cadrul acestei clase sunt implementate funcții care utilizează toate metodele necesare pentru gestionarea datelor unei instanțe Car, împreună cu verificări referitoare la prezența sau absența unei serii de șasiu date. S-au implementat două metode prin care se pot adăuga datele unei mașini pe blockchain. Prima metodă este mai strictă și necesită ca parametru toate datele mașinii, inclusiv: ID-ul utilizatorului, șasiul și trei tupluri care reprezintă structurile „GeneralInfo”, „EngineInfo” și „WheelsInfo”. Cea de-a doua metodă este mai permisivă, constând într-o funcție care instanțiază o variabilă de tip Car fără detalii inițiale. În acest caz, toate atributele variabilei sunt inițializate cu valori implicite, cum ar fi 0 pentru numere sau un string vid pentru șiruri de caractere, iar restul detaliilor pot fi adăugate ulterior, dar numai o singură dată, datorită restricției impuse în metoda de adăugare a detaliilor din clasa Car.

După cum a fost menționat în II.2.2. D, contractul a fost dezvoltat în PyCharm folosind extensia pentru Solidity. Uneori, s-a utilizat Remix datorită interfeței sale intuitive și modului simplu de compilare și implementare, ceea ce facilitează testarea în cazul contractelor de dimensiuni mici.

Toate metodele și logica de organizare au condus la creșterea dimensiunii fișierului binar la peste 24,5 KB, o limită care, odată depășită, face dificilă desfășurarea contractului pe Mainnet [53]. Cu toate acestea, datorită optimizatorului din compilatorul Solidity, contractul poate fi desfășurat cu succes pe blockchain-ul din Ganache. Însă, dacă compilarea împreună cu optimizatorul, rezultă într-un fișier mai mare de 24,5 KB, contractul nu mai poate fi lansat. Prin urmare, pentru dezvoltarea funcționalităților este necesară implementarea unor metode speciale de reducere a dimensiunii sau dezvoltarea mai multor contracte sincronizate între ele. Aceste măsuri vor asigura că dimensiunea contractelor individuale rămâne sub limita de 24,5 KB [54], permițând astfel desfășurarea lor eficientă pe Mainnet.

Despre smart contract este de menționat că valoarea predefinită a prețului de gas în cadrul unei rețele Ethereum locale, utilizând un simulator precum Ganache, este setată la 20 Gwei. Totuși, este posibil să se modifice manual această valoare conform necesităților testelor.

În cadrul procesului inițial al dezvoltării contractului inteligent, prima etapă constă în compilarea acestuia, generând astfel un fișier binar cu o dimensiune de 48 KB. Pentru a realiza implementarea efectivă a contractului în rețeaua blockchain, este necesară executarea unei tranzacții de deploy, care implică o consumație de 4.893 milioane de unități de gas. În comparație, costul de deploy al unui contract vid este de 67 mii de unități de gas. Prin urmare, pentru a putea beneficia de funcționalitățile oferite de contractul inteligent, este esențială finalizarea cu succes a tranzacției de deploy. Această operație implică un cost total de $4.893 * 20 * 10^{15}$ Wei, echivalent cu 0.09786 ETH.

În continuare, după finalizarea tranzacției de lansare a contractului, este posibil să adăugăm o nouă entitate în sistem, cum ar fi o mașină nouă. Operația aceasta, conform constatărilor, implică un cost de 2,65 milioane de unități de gas. În urma adăugării vehiculului, utilizatorul autentificat poate efectua actualizările datelor stocate în contract. Aceste funcții includ schimbarea valorii unui câmp de tip uint32 și adăugarea unei structuri ce conține un uint256, un uint64, un int32 și un string, reprezentând un jurnal al modificărilor. O astfel de metodă care pe lângă inserarea datelor în jurnal, actualizează și kilometrii reprezentați de un uint32, a necesitat o cantitate de 97 mii de unități de gas pentru a fi executată cu succes.

Tabelul 1. Operații contractuale.

<i>Tipul operației</i>	<i>Gas folosit</i>	<i>ETH estimativ</i>
Deploy	4.893 ⁶	0.09786
Adăugare informații generale	2.653 ⁶	0.05306
Modificarea unei caracteristici și adăugarea unui log	97 ³	0.00194
Vizualizare	0	0

După cum se poate observa în Tab. 1, toate metodele, cu excepția celor de tip vizualizare, nu consumă gas deoarece informația nu implică modificări ale stării contractului. Această abordare sporește transparența datelor și facilitează consultarea lor de către utilizatori

Inițial, strategia a fost să se aducă mai multe caracteristici care să ofere o imagine de ansamblu mai completă a mașinii căutate, una dintre aceste caracteristici fiind integrarea imaginilor în interfața grafică. Într-o încercare de a menține conceptul de descentralizare, s-a discutat posibilitatea salvării imaginilor pe blockchain. Cu toate acestea, această metodă prezintă numeroase dezavantaje, cum ar fi timpul mare necesar pentru stocarea sau accesarea imaginilor din rețea și problemele de costuri asociate dimensiunii tot mai mari a stocării.

O soluție care ar fi putut rezolva aceste probleme a fost să se salveze doar *căile* (paths – engl.) imaginilor local, cu planul de a găsi ulterior o metodă descentralizată pentru a le stoca. Bazându-se pe această idee, s-a argumentat că salvarea căilor imaginilor unei mașini într-un vector de string ar fi o soluție practică și simplă pentru început. Această abordare a fost implementată cu succes, însă mai târziu s-a căutat o metodă mai eficientă și mai descentralizată pentru stocarea efectivă a imaginilor.

Acest demers a condus la descoperirea IPFS (InterPlanetary File System) [55], o tehnologie care oferă o modalitate descentralizată de stocare a fișierelor. IPFS este un sistem de fișiere peer-to-peer care permite distribuirea și accesarea conținutului pe internet fără a fi nevoie de un server central. Ideea implementării IPFS ar fi fost să se salveze fișierele imagine pe acea rețea, să se obțină hash-ul acestora și să se adauge hash-ul în blockchain.

Cu toate acestea, în timpul implementării acestui sistem, au apărut probleme semnificative de comunicare cu serverul IPFS, ceea ce a pus în dificultate procesul. S-a încercat și implementarea în TypeScript, dar dificultățile persistente în legătură cu comunicarea au dus la abandonarea ideii de utilizare a IPFS pentru stocarea imaginilor. Aceasta a implicat și renunțarea la conceptul inițial de a salva imagini în blockchain, susținut de argumentul că odată ce o imagine este stocată, ar trebui ștearsă atunci când nu mai corespunde cu aspectul actual al mașinii, pentru a evita eventualele încercări de ștergere a imaginilor pentru ascunderea unor defecte sau probleme.

Aceste considerații au subliniat importanța unor sisteme adiționale de securitate pentru protejarea integrității și veridicității datelor. Totuși, trebuie menționat că aceste sisteme de securitate nu au fost inițial luate în considerare și, din acest motiv, nu au fost implementate. Astfel, dificultățile întâmpinate în integrarea IPFS și în gestionarea imaginilor în blockchain au dus la renunțarea la aceste soluții, având în vedere riscurile asociate gestionării și actualizării corecte a datelor, în special în contextul schimbărilor sau modificărilor ulterioare ale informațiilor stocate.

II.2.4. Rețele neuronale

Rețelele neuronale din această lucrare au fost gândite cu scopul de a ajuta la o scanare rapidă a datelor în detrimentul unor seturi de date generate care respectă câteva condiții impuse.

Prima rețea analizează distanța parcursă în kilometri și anul de fabricație al vehiculului.

Astfel, se consideră că în medie sunt parcurși 16.000 de kilometri pe an, iar ca limite extreme s-au stabilit valorile de 14.000 și 20.000 de kilometri. În consecință, dacă numărul de kilometri parcurși într-un an este mai mic de 14.000 sau mai mare de 20.000, vehiculul nu se mai încadrează în intervalul ales.

În cadrul acestei rețele, cu cât distanța parcursă este mai îndepărtată de acest interval, cu atât se va încadra mai bine în clasa specificată pentru această problemă, care reprezintă de fapt o abatere de la media stabilită.

Structura concretă a rețelei constă într-un strat de intrare format din 2 neuroni, urmat de o normalizare Batch, două straturi ascunse, fiecare având 16 neuroni, și un strat de ieșire cu un singur parametru, care se pot vedea în Anexa 2.

Cu ajutorul datelor de validare, s-a putut ajunge mai ușor la arhitectura finală. Astfel, s-a observat că un număr mai mare de straturi sau un număr mai mare de neuroni pe strat pentru această problemă nu aduce o îmbunătățire, ci dimpotrivă, în cazul în care s-a folosit un număr mai mare de 32 de neuroni, de exemplu, rețeaua a suferit probleme de *supraînvățare*, unde devine prea specializată pentru datele de antrenament și nu generalizează bine pentru datele noi. Aceasta a condus la adoptarea unui număr mai redus de neuroni. În cazul straturilor, un număr mai mare ducea la zgomot și la dificultăți în învățarea concretă a regulilor problemei.

Deoarece datele de intrare constau în 2 numere care pot avea diferențe semnificative între ele, s-a căutat o metodă de normalizare a acestora. Inițial, s-a încercat utilizarea mediei setului de date pentru a normaliza restul datelor, însă această abordare nu a dus la o îmbunătățire semnificativă. În continuare, s-a experimentat cu normalizarea folosind media și dispersia, dar rezultatele nu au fost satisfăcătoare până când s-a adoptat normalizarea pe loturi (batch).

În timpul antrenării, funcția Batch normalizează intrările utilizând media și deviația standard a lotului curent, conform formulei (5).

$$lot_i = lot_i - \frac{\sum_{j=0}^n lot_{ij}}{n} - \frac{n}{\sqrt{\sigma(lot_i) + 0.001}} \#(5)$$

S-a utilizat funcția BatchNormalization() din biblioteca Keras în locul uneia implementate manual, deoarece era necesară stocarea mediei și a dispersiei în fișiere externe pentru a putea fi utilizate în momentul predicției pe setul de date de intrare. Astfel, biblioteca Keras gestionează automat stocarea acestor date în momentul în care sunt calculate, împreună cu ponderile rezultate în urma antrenării.

La predicție, se utilizează media mediilor și a deviațiilor salvate în timpul antrenării, această abordare fiind justificată de discrepanțele semnificative dintre valorile parametrilor.

Tabelul 2. Rezultate rețea 1.

<i>An fabricație</i>	<i>Kilometrii făcuți</i>	<i>Clasa</i>	<i>Predicție</i>
2010	15000	1	0.99998
2011	15000	1	0.99996
2015	128000	0	0.42677
2020	80000	1	0.63310
2000	312000	0	0.11798
1999	320000	0	0.22448
1960	1152000	0	0.33453*10 ⁷

A doua rețea analizează numărul de întrețineri pe an și numărul de vânzări pe an pentru a deduce regulile pe baza cărora setul de date a fost structurat. Configurația rețelei este următoarea: un strat de intrare cu 2 parametri, o normalizare Batch care a adus îmbunătățiri minore, dat fiind că datele sunt apropiate, un strat ascuns cu 16 neuroni și un strat de ieșire cu 2 neuroni, care se poate vizualiza în Anexa 2. Astfel, prima clasă indică posibilitatea unor probleme tehnice viitoare, notată cu 1, în timp ce a doua clasă reprezintă posibilitatea unor daune tehnice ascunse, notată cu 2. Regulile care guvernează atribuirea claselor sunt următoarele:

- Dacă numărul de vânzări este mai mare de 0.2 (adică o dată la 5 ani), atunci se atribuie clasa 2.
- Dacă numărul de întrețineri este mai mare de 2, atunci se atribuie clasa 1.
- Dacă numărul de întrețineri este mai mic de 0.5, atunci se atribuie clasa 2.

Tabelul 3. Rezultate rețea 2.

<i>Mentenanțe</i>	<i>Vânzări</i>	<i>Clasa</i>		<i>Predicție</i>	
		1	2	1	2
2	0.2	0	0	0.36997	0.45592
1.9	0.2	0	0	0.38954	0.23971
1	1.2	1	0	1	$0.65248 \cdot 10^{-4}$
2	0.9	1	0	1	0.52711
2.2	0.2	0	1	0.33649	0.85664
0.2	0.2	1	0	0.96415	$0.324109 \cdot 10^{-7}$

A treia rețea neuronală operează pe baza a trei straturi de intrare, fiecare cu câte trei parametri, urmată de trei straturi ascunse compuse din câte 16 neuroni, și un strat de ieșire format din doi parametri, prezentată în Anexa 2. Scopul ei este să analizeze numărul de accidente, reparații și daune înregistrate pe parcursul unui an. Setul de date asociat acestui proces include două clase distincte: clasa 1, care reprezintă daune tehnice ascunse și clasa 2, care reprezintă daune majore. Condițiile guvernante care conduc procesul de clasificare sunt după cum urmează:

- Dacă numărul de accidente este mai mare decât 1, atunci setul se atribuie atât clasei 1, cât și clasei 2.
- Dacă nivelul de daune depășește 3, atunci se atribuie clasei 2.
- Dacă diferența dintre numărul de reparații și suma dintre numărul de accidente și nivelul de daune este mai mare decât 2, atunci se atribuie clasei 1.
- Dacă numărul de accidente este mai mare decât numărul de reparații, atunci se atribuie clasei 2.
- Dacă nivelul de daune este mai mare decât numărul de reparații, atunci se atribuie clasei 1.

Tabelul 4. Rezultate rețea 3.

<i>Accidente</i>	<i>Daune</i>	<i>Reparații</i>	<i>Clasa</i>		<i>Predicție</i>	
			1	2	1	2
0.1	1	2	0	0	$0.1282 \cdot 10^{-7}$	$0.8122 \cdot 10^{-3}$
1.1	1	2	1	1	0.8946	0.7174
0.9	3.1	5	1	0	0.7721	$0.3531 \cdot 10^{-1}$
0.9	2	5	0	1	$0.1361 \cdot 10^{-2}$	0.6291
0.9	0	0.8	1	0	0.9865	$0.1711 \cdot 10^{-1}$
0.8	1.1	0.9	0	1	$0.1538 \cdot 10^{-4}$	0.9118

Procesul de validare a jucat un rol important și în dezvoltarea acestei rețele, oferind o

perspectivă clară asupra performanței. În etapa inițială, s-a constatat că un număr redus de straturi nu este suficient pentru a învăța regulile complexe implicate în problema dată. Această observație a fost fundamentală în decizia de a crește numărul de straturi pentru a permite rețelei să captureze mai bine interacțiunile și dependențele dintre datele de intrare.

Odată cu adăugarea de straturi suplimentare, s-au înregistrat îmbunătățiri semnificative în metricile utilizate pentru evaluarea performanței rețelei. Aceste metrici au furnizat indicii esențiale în determinarea numărului optim de straturi care să asigure o echilibrare între complexitatea modelului și capacitatea acestuia de a generaliza corect pe date noi.

În procesul de dezvoltare a acestor rețele neuronale, au fost urmărite diverse metrici pentru fiecare model. Coloanele „Batch” din Tab. 5 reflectă calculele efectuate pe lotul curent de date, în timp ce coloanele „Val” reflectă calculele efectuate pe setul de date de validare.

Tabelul 5. Metrici finale.

Model	Accuracy		Loss		MSE	
	Batch	Val	Batch	Val	Batch	Val
Kilometri Ani	0.8784	0.9563	0.2303	0.1336	0.0756	0.0355
Mentenanțe Vânzări	0.9419	0.9290	0.1198	0.0576	0.0375	0.0137
Accidente Daune Reparații	0.8421	0.8300	0.0496	0.0502	0.0140	0.0155

Concluzii

În concluzie, lucrarea propusă investighează integrarea tehnologiilor emergente, cum ar fi blockchain-ul și inteligența artificială, pentru a dezvolta o soluție transparentă pentru gestionarea datelor în contextul tranzacțiilor auto. Prin utilizarea contractelor inteligente în cadrul unui blockchain, se asigură o evidență imutabilă și sigură a informațiilor legate de vehicule, eliminând nevoia unui intermediar și creând un mediu de încredere pentru părțile implicate. De asemenea, integrarea rețelelor neuronale în analiza datelor oferă o perspectivă avansată asupra evenimentelor și incidentelor legate de autovehicule, contribuind la luarea deciziilor informate.

Această abordare nu doar îmbunătățește eficiența procesului de verificare și gestionare a datelor, dar deschide și noi posibilități pentru dezvoltarea ulterioară a aplicațiilor în domenii variate. Astfel, lucrarea reprezintă un pas semnificativ în direcția creării unui mediu digital mai transparent și mai eficient pentru industria auto, contribuind la facilitarea tranzacțiilor și la creșterea încrederii între părțile implicate.

Aplicația dezvoltată denotă o platformă care oferă utilizatorilor acces la informații veridice și complete despre vehicule, fără a depinde de un intermediar. Aceasta contribuie semnificativ la creșterea gradului de încredere în tranzacțiile auto prin reducerea timpului și a costurilor asociate eventualelor probleme ulterioare. Prin accesul la istoricul complet al mașinii, atât cumpărătorii, cât și vânzătorii beneficiază de o transparență sporită și de o tranzacție mai facilă și mai sigură. Astfel, platforma dezvoltată nu doar că susține luarea unor decizii informate, dar și îmbunătățește experiența generală a utilizatorilor în piața vehiculelor second-hand.

Chiar dacă nivelul de securitate este redus, concluziile obținute indică faptul că blockchain-ul reușește să integreze atât caracteristica de transparență, prin accesul facil la date și informații referitoare la tranzacții, cât și cea de imutabilitate, prin utilizarea unui contract ce impune restricții în ceea ce privește ștergerea datelor. Modelele de inteligență artificială implementate au atins un nivel de performanță satisfăcător pentru condițiile date, furnizând rezultate care pot asista persoanele neavizate în procesul decizional al unei tranzacții legate de un vehicul.

De asemenea, rezultatele obținute implică realizarea unei interfețe grafice intuitive, care permite utilizatorilor să acceseze rapid informațiile necesare fără a necesita o procesare complexă pentru a le înțelege. În plus, utilizarea modelelor de învățare automată a permis analiza setului de date și generarea de rezultate relevante, ceea ce a facilitat luarea de decizii informate. Separarea tipurilor de utilizatori a permis o gestionare clară a contribuțiilor și a accesului la informații, astfel încât doar anumite persoane să poată gestiona datele sau să editeze starea altor utilizatori. Totodată, această separare a structurii în componente oferă o serie de avantaje, întrucât pentru a reutiliza interfața grafică în alte domenii care necesită metode de verificare a produselor, este necesară înlocuirea unui număr redus de elemente. Modularitatea la nivel de aplicație reprezintă, la rândul său, un avantaj semnificativ, deoarece permite adăugarea facilă a unor noi module ce extind sau aduc noi funcționalități.

Această separare atât la nivel de module cât și la nivel de componente nu doar că simplifică procesul de adaptare și implementare a soluției în diverse industrii, dar contribuie și la reducerea costurilor și a timpului de dezvoltare, asigurând totodată o flexibilitate crescută în fața cerințelor variabile ale pieței.

În vederea îmbunătățirii acestei soluții, se recomandă adăugarea unui set mai variat de date, inclusiv imagini stocate în IPFS, și crearea unui al doilea contract inteligent pentru gestionarea utilizatorilor. În ciuda obiectivului de transparență, nu oricine ar trebui să aibă acces la modificarea datelor, astfel că implementarea unui sistem blockchain cu noduri, care utilizează mecanisme mai avansate de validare și verificare, ar putea spori semnificativ nivelul de securitate și fiabilitate al

soluției. Aceste îmbunătățiri ar asigura nu doar o protecție mai mare a datelor, dar și o mai bună gestionare a accesului și modificărilor, consolidând încrederea utilizatorilor în acest sistem.

Bibliografie

- [1] ***, Angular Framework, <https://angular.dev/>, ultima accesare: 26/06/2024.
- [2] ***, Python Programming Language, <https://www.python.org>, ultima accesare: 26/06/2024.
- [3] ***, Solidity programming language, <https://soliditylang.org>, ultima accesare: 26/06/2024.
- [4] ***, Ganache-UI, <https://github.com/trufflesuite/ganache-ui>, ultima accesare: 26/06/2024.
- [5] ***, Npm, <https://www.npmjs.com/>, ultima accesare: 26/06/2024.
- [6] ***, NodeJs, <https://nodejs.org/en>, ultima accesare: 26/06/2024.
- [7] ***, Solidity Compiler <https://docs.soliditylang.org/en/latest/installing-solidity.html>, ultima accesare: 26/06/2024.
- [8] ***, Autodna, <https://www.autodna.ro/>, ultima accesare: 26/06/2024.
- [9] ***, Rarom, <https://www.rarom.ro/>, ultima accesare: 26/06/2024.
- [10] ***, Vincheck, <https://www.vincheck.ro/>, ultima accesare: 26/06/2024.
- [11] ***, CarVertical, <https://www.carvertical.com/>, ultima accesare: 26/06/2024.
- [12] ***, Smart Contracts, <https://www.ibm.com/topics/smart-contracts>, ultima accesare: 26/06/2024.
- [13] ***, Blockchain, <https://www.ibm.com/topics/blockchain>, ultima accesare: 26/06/2024.
- [14] ***, What is blockchain, <https://www.oracle.com/ro/blockchain/what-is-blockchain/>, ultima accesare: 26/06/2024.
- [15] F. Casino, E. Politou, E. Alepis and C. Patsakis, "Immutability and Decentralized Storage: An Analysis of Emerging Threats," in IEEE Access, vol. 8, pp. 4737-4744, 2020, doi: 10.1109/ACCESS.2019.2962017.
- [16] E. Politou, F. Casino, E. Alepis and C. Patsakis, "Blockchain Mutability: Challenges and Proposed Solutions," in IEEE Transactions on Emerging Topics in Computing, vol. 9, no. 4, pp. 1972-1986, 1 Oct.-Dec. 2021, doi: 10.1109/TETC.2019.2949510.
- [17] H. S. Kim and K. Wang, "Immutability Measure for Different Blockchain Structures," 2018 IEEE 39th Sarnoff Symposium, Newark, NJ, USA, 2018, pp. 1-6, doi: 10.1109/SARNOF.2018.8720496.
- [18] ***, Ethereum, <https://ethereum.org/en/>, ultima accesare: 26/06/2024.
- [19] R. K. Kaushal, N. Kumar, S. N. Panda and V. Kukreja, "Immutable Smart Contracts on Blockchain Technology: Its Benefits and Barriers," 2021 9th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), Noida, India, 2021, pp. 1-5, doi: 10.1109/ICRITO51393.2021.9596538.
- [20] Mehdi Salehi, Jeremy Clark, and Mohammad Mannan. 2023. Not so Immutable: Upgradeability of Smart Contracts on Ethereum. In Financial Cryptography and Data Security. FC 2022 International Workshops: CoDecFin, DeFi, Voting, WTSC, Grenada, May 6, 2022, Revised Selected Papers. Springer-Verlag, Berlin, Heidelberg, 539–554. https://doi.org/10.1007/978-3-031-32415-4_33.
- [21] ***, Decentralized Applications (dApps): Definition, Uses, Pros and Cons, <https://www.investopedia.com/terms/d/decentralized-applications-dapps.asp>, ultima accesare: 26/06/2024.
- [22] Dietrich F., Turgut A., Palm D., Louw L. (2020). Smart Contract-Based Blockchain Solution to Reduce Supply Chain Risks. In: Lalic, B., Majstorovic, V., Marjanovic, U., von Cieminski, G., Romero, D. (eds) Advances in Production Management Systems. Towards Smart and Digital Manufacturing. APMS 2020. IFIP Advances in Information and Communication Technology, vol 592. Springer, Cham. https://doi.org/10.1007/978-3-030-57997-5_20.
- [23] Aldyafiah, I.M., Zhao W., Upadhyay H.; Lagos L., The Design and Implementation of a Secure Datastore Based on Ethereum Smart Contract. Appl. Sci. 2023, 13, 5282. <https://doi.org/10.3390/app13095282>.

- [24] ***, Gas (Ethereum): How Gas Fees Work on the Ethereum Blockchain, <https://www.investopedia.com/terms/g/gas-ethereum.asp>, ultima accesare: 26/06/2024.
- [25] ***, What is EIP-1559?, <https://consensys.io/blog/what-is-eip-1559-how-will-it-change-ethereum>, ultima accesare: 26/06/2024.
- [26] Florin Leon, Inteligență artificială – note de curs, http://florinleon.byethost24.com/curs_ia.html, ultima vizitare: 26/06/2024.
- [27] Zhang Peter. (2000). Neural Networks for Classification: A Survey. Systems, Man, and Cybernetics, Part C: Applications and Reviews, IEEE Transactions on. 30. 451 - 462. 10.1109/5326.897072.
- [28] ***, Data binding, <https://angular.jp/guide/binding-overview>, ultima accesare: 26/06/2024.
- [29] ***, Sweetalert2, <https://sweetalert2.github.io/>, ultima accesare: 26/06/2024.
- [30] ***, Structural directives , <https://angular.jp/guide/structural-directives>, ultima accesare: 26/06/2024.
- [31] ***, Flask Framework, <https://flask.palletsprojects.com/en/3.0.x/>, ultima accesare: 26/06/2024.
- [32] ***, Tensorflow library, <https://www.tensorflow.org/>, ultima accesare: 26/06/2024.
- [33] ***, Keras, <https://keras.io/>, ultima accesare: 26/06/2024.
- [34] ***, Web3, <https://web3py.readthedocs.io/en/stable/>, ultima accesare: 26/06/2024.
- [35] ***, Solcx, <https://pypi.org/project/py-solc-x/>, ultima accesare: 26/06/2024.
- [36] ***, SQLAlchemy library, <https://www.sqlalchemy.org/>, ultima accesare: 26/06/2024.
- [37] ***, PyJWT, <https://pyjwt.readthedocs.io/en/stable/>, ultima accesare: 26/06/2024.
- [38] ***, RFC 7519, <https://datatracker.ietf.org/doc/html/rfc7519>, ultima accesare: 26/06/2024.
- [39] Paul P., Aithal P. S., Saavedra R., Ghosh Surajit, Blockchain Technology and Its Types—A Short Review (December 26, 2021). International Journal of Applied Science and Engineering (IJASE), 9(2), 189-200. (2021). ISSN: 2321-0745. , Available at SSRN: <https://ssrn.com/abstract=4050933>.
- [40] ***, Three-tier Architecture, <https://www.ibm.com/topics/three-tierarchitecture>, ultima accesare: 26/06/2024.
- [41] Alexandru Achip, Programare orientată pe servicii – note de laborator, ***, ultima accesare: 26/06/2024.
- [42] ***, Spring Framework, <https://spring.io/projects/spring-framework>, ultima accesare: 26/06/2024.
- [43] ***, Java Programming Language, <https://www.java.com/en/>, ultima accesare: 26/06/2024.
- [44] ***, Web3j, <https://docs.web3j.io/4.11.0/>, ultima accesare: 26/06/2024.
- [45] ***, Remix, <https://remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljsn-v0.8.18+commit.87f61d96.js>, ultima accesare: 26/06/2024.
- [46] ***, Visual Studio Code, <https://code.visualstudio.com/>, ultima accesare: 26/06/2024.
- [47] ***, Truffle Suite, <https://archive.trufflesuite.com/>, ultima accesare: 26/06/2024.
- [48] ***, Pycharm, <https://www.jetbrains.com/pycharm/>, ultima accesare: 26/06/2024.
- [49] ***, Solidity Plugin Pycharm, <https://plugins.jetbrains.com/plugin/9475-solidity>, ultima accesare: 26/06/2024.
- [50] ***, Binary loss function, <https://arize.com/blog-course/binary-cross-entropy-log-loss/>, ultima accesare: 26/06/2024.
- [51] Florin Leon, Învățare automată – note de curs, http://florinleon.byethost24.com/curs_ia.html, ultima accesare: 26/06/2024.
- [52] ***, Sigmoid Function in Artificial Neural Networks, <https://www.analyticsvidhya.com/blog/2023/01/why-is-sigmoid-function-important-in-artificial-neural-networks/>, ultima accesare: 26/06/2024.
- [53] ***, Mainnet, <https://academy.binance.com/en/glossary/mainnet>, ultima accesare: 26/06/2024.
- [54] Vitalik Buterin, EIP-170: Contract code size limit, <https://eips.ethereum.org/EIPS/eip-170>, ultima accesare: 26/06/2024.

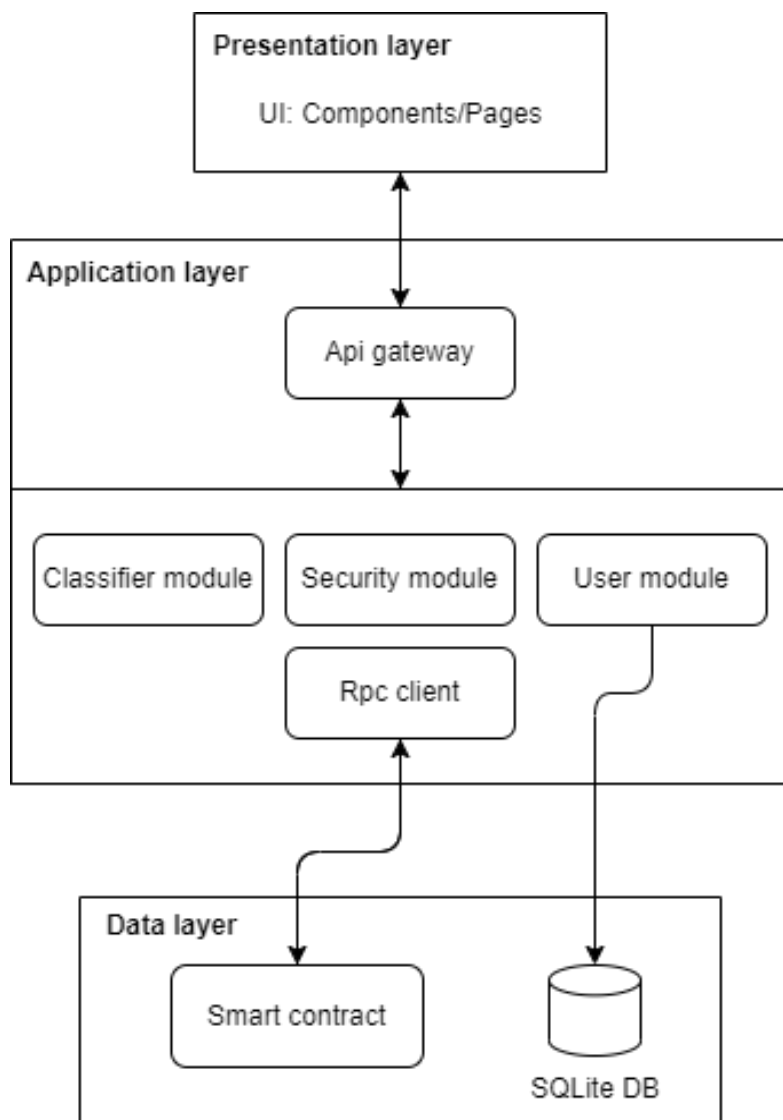
[55] ***, IPFS, <https://ipfs.tech/>, ultima accesare: 26/06/2024

Anexe

Anexa 1. Schema arhitecturii

A1. este schema arhitecturala a aplicației, iar elementele acesteia sunt după cum urmează:

- Dreptunghiurile mari, fără colturi rotunjite sunt nivelele arhitecturii.
- Dreptunghiurile mici a căror colturi sunt rotunjite reprezintă module ce îndeplinesc un scop.
- Săgețile reprezintă comunicarea între doua componente.
- Obiectul notat cu „SQLite DB” este o baza de date.



A.1. Arhitectura aplicației.

Anexa 2. Secvențe din Python

Codul următor reprezintă o clasă prin care e realizată interacțiunea cu o tabelă SQL cu ajutorul SQLAlchemy, astfel:

- `__tablename__` : definește numele tabelii.
- `id`: este cheia primară prin care se identifica un rând.
- `username, name, password, company`: sunt numele de utilizator, numele real, parola acestuia și compania.
- `Is_active`: este o valoare booleană ce semnifică dacă utilizatorul are sau nu acces la funcția de autentificare.
- `Role`: este un string care identifică utilizatorii cu rol „USER” sau „ADMIN”.

```
class User(Base):
    __tablename__ = 'users'
    id = Column(Integer, primary_key=True)
    username = Column(String)
    name = Column(String)
    password = Column(String)
    is_active = Column(Boolean, default=True)
    company = Column(String)
    role = Column(String)
```

A.2.1. Clasă ce definește structura tabelii SQLite.

Clasa de mai jos moștenește un Enum și este folosit pentru a identifica mai ușor cheile din “secrets.txt”, astfel:

- `TOKEN_KEY`: identifică secretul folosit în generarea token-urilor.
- `PRIVATE_KEY`: este cheia privată a contului din blockchain folosită pentru semnare.
- `ABI_FILE_HASH` și `ABI_FILE_PATH`: prima este valoarea hash a fișierului .ABI și a doua este calea acelui fișier.
- `ISS`: este adresa URL a serverului de Flask folosit în generarea token-urilor.
- `MINUTES_EXP`: este timpul exprimat în minute care reprezintă durata până expiră semnătura token-ului.
- `CONTRACT1_HASH` și `CONTRACT2_HASH`: sunt valorile hash a fișierelor ce reprezintă contractul inteligent, folosite pentru detecția unor posibile schimbări în documente.
- `SOLC_PATH`: este calea către compilatorul Solidity.

```
class Keys(Enum):
    TOKEN_KEY = "TOKEN_KEY"
    CONTRACT_ADDRESS = "CONTRACT_ADDRESS"
    PRIVATE_KEY = "PRIVATE_KEY"
    ABI_FILE_HASH = "ABI_FILE_HASH"
    ABI_FILE_PATH = "ABI_FILE_PATH"
    ISS = "ISS"
    MINUTES_EXP = "MINUTES_EXP"
    CONTRACT1_HASH = "CONTRACT1_HASH"
    CONTRACT2_HASH = "CONTRACT2_HASH"
    SOLC_PATH = "SOLC_PATH"
```

A.2.2 Clasă ce deține cheile secrete.

Obiectele de mai jos reprezintă arhitectura unor modele de rețele neuronale, după cum urmează:

- Prima rețea are un strat de intrare cu doi neuroni, o normalizare de tip Batch, două straturi ascunse „fully connected” a câte 16 neuroni fiecare și un strat de ieșire cu un singur neuron.

```
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=(2,)),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
]).
```

A.2.3 Structura primului model.

- A doua rețea are un strat de intrare cu doi neuroni, o normalizare de tip Batch, un strat ascuns „fully connected” de 16 neuroni și un strat de ieșire cu doi neuroni.

```
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=(2,)),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(2, activation='sigmoid')
])
```

A.2.4 Structura celui de-al doilea model.

- A treia rețea are un strat de intrare cu trei neuroni, trei straturi ascunse „fully connected” a câte 16 neuroni fiecare și un strat de ieșire cu doi neuroni.

```
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(input_shape=(3,)),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(16, activation='relu'),
    tf.keras.layers.Dense(2, activation='sigmoid')
])
```

A.2.5 Structura celui de-al treilea model.

Anexa 3. Secvențe din Solidity

Structurile de mai jos reprezintă un mod de a încapsula separat informațiile generale, despre motor și a roților unei mașini. Comentariile din partea dreaptă a proprietății reprezintă un exemplu de valoare pe care o poate lua acel atribut.

```
struct GeneralInfo {
    string chassisNumber; //VF10046132340
    uint32 manufacturingYear; //2018
    string manufacturer; //Renault
    string model; //Kadjar
    string bodyType; //caroserie: SUV
    string gearbox; //Manuală/automată/One_gear
    string color; //negru
    uint32 noSeats; //2
    uint32 noDoors; //4
    uint32 noKm; //100000
    string transmission; // FWD/RWD
}
```

A.3.1. Structură ce deține datele generale ale mașinii.

```
struct EngineInfo{
    string serial;
    string liters;
    uint32 horsePower;
    string fuelType; // diesel/benzină/electric/hibrid
}
```

A.3.2. Structură ce deține datele despre motorul mașinii.

```
struct WheelsInfo{
    uint32 noWheels;
    uint32 diameter;
    uint32 width;
}
```

A.3.3 Structură ce deține datele despre roțile mașinii.

Declarațiile de tip enumerație de mai jos reprezintă tipurile de modificări și evenimente posibile, care pot fi adăugate în istoricul unei mașini.

```
enum Part {
    Odometer,
    Engine,
    Transmission,
    Suspension,
    Brakes,
    Wheels,
    Body,
    Interior,
    Exterior,
    Electronics,
    Other
}
```

A.3.4 Enum pentru tipurile de modificări.

```
enum EventName {  
    Crash,  
    Theft,  
    Damage,  
    TechnicalRevision,  
    ITP,  
    Purchase,  
    Sale,  
    Service,  
    Maintenance,  
    Register,  
    Other  
}
```

A.3.5 Enum pentru tipurile de evenimente.